

MASTER

Model checking for analysis of BPMN models

van Uden, Noah A.

Award date:
2025

[Link to publication](#)

Disclaimer

This document contains a student thesis (bachelor's or master's), as authored by a student at Eindhoven University of Technology. Student theses are made available in the TU/e repository upon obtaining the required degree. The grade received is not published on the document as presented in the repository. The required complexity or quality of research of student theses may vary by program, and the required minimum study period may vary in duration.

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain

Take down policy

If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.



Department of Mathematics and Computer Science
Formal System Analysis

Model checking for analysis of BPMN models

Master Thesis Report

Noah van Uden

Supervisors:

Dr. Ir. Tim Willemse

Dr. Debjyoti Bera

Dr. Raúl Monti

April-2025

Abstract

Model-Based Testing (MBT) is a promising technology to extend test coverage beyond manually crafted tests, by automatically creating more, longer, and more diverse test cases, with less effort, from abstract models of the required behavior. When the models used for MBT are correct, the test cases are provably valid as well. TNO-ESI and ASML have been working on realizing MBT. With MBT, the hope is that the amount of time spent on creating System-of-System tests is reduced, while drastically increasing test coverage. Currently, the correctness of the models used for MBT is not analyzed. In this thesis, the aim is to introduce model checking for analysis of BPMN models.

For model checking, mCRL2 is introduced as a verification back-end for the formal models used for MBT. Additionally, an easy-to-use property language is introduced and the results of model checking are translated back to the notation of BPMN models.

AI-assisted technology is not used in preparation of this work.

Contents

1	Introduction	3
1.1	Problem Statement	4
1.2	Findings	6
1.3	Thesis structure	6
1.4	Artifacts	7
2	Mathematical preliminaries	8
3	Colored Petri Nets	11
3.1	Petri Nets	11
3.2	Colored Petri Nets	12
3.3	Semantics	14
4	Business Process Modeling & Notation	17
4.1	BPMN syntax	18
4.2	Data in BPMN models	20
4.3	A Petri Net semantics of BPMN models	21
5	Translating CPNs to mCRL2	23
5.1	Definitions	23
5.2	Translating CPN models	24
5.3	Semantics and equivalence	25
5.4	Implementation of the translation	28
6	Properties for Model Checking	31
6.1	Model checking	31
6.2	Related work	33
6.3	Property templates	34
7	Results of Model Checking	38
7.1	Counterexamples	38

7.2	Message Sequence Charts	40
7.3	Simulation	46
8	Case Study	48
8.1	BPMN model	48
8.2	Properties to check	49
8.3	Evaluation	52
9	Conclusion	53
10	Future Research	55
10.1	Efficiency of model checking	55
10.2	Steering test case generation	56
10.3	Assertions in BPMN models	57
	APPENDICES	60
A	Full Translation	61
B	Property Templates	65
C	Simulation	72

Chapter 1

Introduction

Testing is important to increase the quality and reliability of software systems, but writing test cases by hand is tedious and error prone. Model-Based Testing (MBT) is a promising technology to extend the test coverage beyond manually crafted tests. With MBT a system under test (SUT) is tested against an abstract model of its required behavior. Because test cases are generated automatically, it is possible to generate more, longer, and more diverse test cases, with less effort. Additionally, the test cases are provably valid [1].

Within the Matala project, TNO-ESI and ASML have been working on realizing MBT for Cross Product Integration. Traditionally, creating test cases for these System-of-System scenarios has been very time consuming. With MBT the hope is that after an investment in developing the MBT technology and creating models, there is a drastic reduction in the amount of time spent creating tests, while drastically increasing the test coverage. This can both reduce the costs of testing systems, as well as reduce the risk of system failures.

The effectiveness of MBT depends on the correctness of the models [1]. However, within the Matala project, there are currently no methods for analyzing the correctness of the models beyond looking at the static structure. For analyzing models, the aim is to introduce model checking. With model checking, the modeler will be able to specify formal properties and check whether the model adheres to these properties, as shown in Figure 1.1. With this technique, the goal is to increase the confidence that the model behaves as the modeler had intended.

Currently, the models are specified in Business Process Model and Notation (BPMN) [2] language, where the semantics are defined in terms of

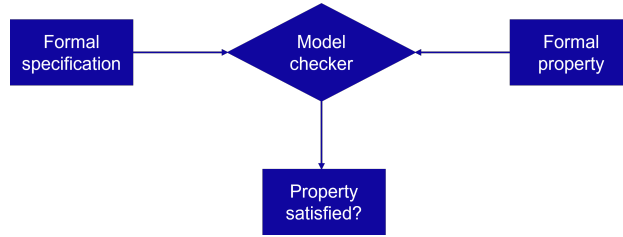


Figure 1.1: Model checking

a mapping to Colored Petri Nets (CPN) [3]. To use model checking, an additional translation to a verification back-end must be introduced. Additionally, modelers should be able to express properties and interpret the results of model checking on BPMN level.

1.1 Problem Statement

Since Model-Based testing relies heavily on the correctness of the behavior of the underlying model [1], model checking would be a valuable addition to the Matala project. Using model checking, the confidence in the correctness of the BPMN models developed at ASML would be significantly improved. And thus also the confidence in Model-Based Testing.

It is important that model checking can be used efficiently, thus a modeler should be able to analyse a model within a reasonable amount of time with easy-to-use tools. Additionally, model checking should be effective, thus the results should be provably correct, and the results should be easy to interpret by a modeler.

This gives rise to the following question:

RQ. *How to efficiently and effectively use model checking to analyse BPMN models?*

To answer this question, three research questions are introduced and explained further on in this section. How these questions combine to answer the main question is shown in Figure 1.2.

1.1.1 Research Question 1

To use the BPMN models for model checking, the model first needs to be translated to a specification that can be interpreted by a model checker. First, a rigorous definition of the CPN semantics of BPMN must be noted down. Then, a translation from the CPN semantics to mCRL2 can be defined.

The first important aspect is the efficiency and effectiveness of the translation. It is important that the translation is efficient, i.e., that it leads to an optimized mCRL2 specification, to prevent unnecessarily long waiting time when model checking. Additionally, it is important that the translation is correct, i.e., that properties hold in a translated model if and only if the properties hold in the original BPMN model.

This leads to the first research question:

RQ 1. *Can a correct and efficient translation be defined from BPMN specifications to mCRL2 specifications to allow for model checking?*

1.1.2 Research Question 2

To use mCRL2 for model checking, properties can be specified in the modal μ -calculus. Even though the μ -calculus is a very expressive language, it is not very intuitive. Deciphering nested fixed points can take a lot of effort. Additionally, properties express desired behavior in the BPMN model, thus it is desirable to be able to specify properties in notation similar to that of BPMN models. This results in the notation of the specified property differing from the notation of the model that is given as input to the model checker.

RQ 2. *Can formal properties be expressed in an easy-to-understand way?*

1.1.3 Research Question 3

Model checking does not only lead to a Yes/No answer to the question: “Does the model satisfy a specific property?”. In case a property is satisfied model checking will also result in a witness, and if the property is not satisfied it will result in a counterexample. Counterexamples and witnesses are sub models of the model given as input to the model checker [4]. Thus, if a property is not satisfied, a counterexample might not be enough to give the modeler an intuitive sense why the original BPMN model is not valid.

To profit from the benefits of model checking it is vital to be able to present counterexamples or witnesses in a way that is understandable by the creator of the BPMN model. For such a translation, there are several aspects to consider.

BPMN models are hierarchical, during translation they are flattened in non-hierarchical CPN models. Thus, the notion of hierarchy has disappeared by the time the models are translated into mCRL2 specifications. Also, the translation from counterexamples and witnesses can be done in one, or in two steps, first translating it to CPN specifications and then to BPMN level.

This leads to the second research question:

RQ 3. *Can the results of model checking BPMN models be presented in an understandable way to the modeler?*

An overview of how the research questions interact with each other is shown in Figure 1.2.

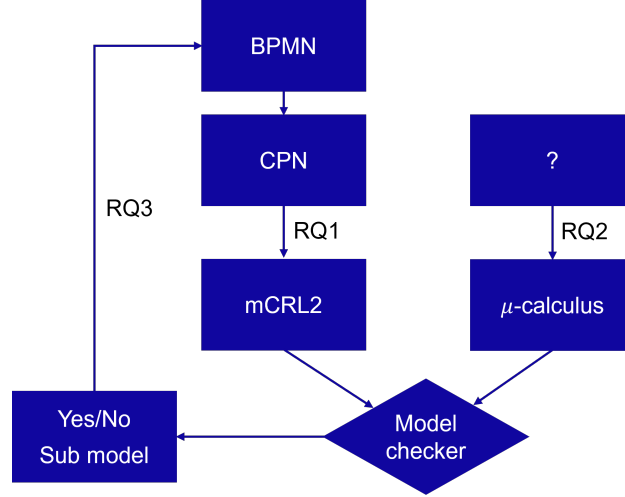


Figure 1.2: Overview of the research questions

1.2 Findings

In this thesis, a method for using model checking for BPMN models is laid out. Using the sound translation, as discussed in Chapter 5, and the property templates, as discussed in Chapter 6, the mCRL2 model checker can be used. The results can then be presented to the user as Message Sequence Charts or simulated as BPMN models.

1.3 Thesis structure

This thesis is structured as follows. Some necessary mathematical definitions are given in Chapter 2. Next, Petri Nets and Colored Petri Nets are introduced in Chapter 3. Then, the background of BPMN is discussed in Chapter 4. In Chapter 5, mCRL2 is defined and research question one is answered. Then, in Chapters 6 and 7 research questions two and three are discussed. In Chapter 8 a case study to evaluate the research questions is done. And

finally in Chapters 9 and 10, the thesis is concluded, and possible future research is discussed.

1.4 Artifacts

All of the code written for the thesis is stored as a branch of the OfflineMBT repository¹. The readme file contains an installation guide for the main branch of the repository. After following the installation guide, an Eclipse project will open, from there the branch should be switched to ‘mcrl2-model-checking’. The solution can be built by running the ‘Generate all Language Infrastructures’ launch file, then the project can be opened.

¹<https://github.com/dbera/OfflineMBT/tree/mcrl2-model-checking>

Chapter 2

Mathematical preliminaries

In the following chapters some modelling concepts are formally defined. To define the syntax and semantics of these concepts precisely, some definitions and their notation need to be introduced. First, bags together with the specific notation for bags used in this document are introduced. Then, expressions and how expressions are evaluated is introduced. Lastly, Labeled Transition Systems are introduced.

Definition 1 (Bag). *A bag m over a non-empty set S is a function $m : S \rightarrow \mathbb{N}$. Where for each $s \in S$, $m(s)$ is the number of appearances of s in m , also called the coefficient of s in m .*

Each unique item in a bag is written with its *coefficient*. So, the bag containing only the number 1 occurring three times is written as $\{3'1\}$.

Membership, addition, comparison, size, summation and subtraction are defined as follows, where m and m_i for all $i \in \mathbb{N}$, are bags over S :

1. $\forall s \in S : s \in m \Leftrightarrow m(s) > 0$.
2. $\forall s \in S : (m_1 \cup m_2)(s) = m_1(s) + m_2(s)$.
3. $m_1 \subseteq m_2 \Leftrightarrow \forall s \in S : m_1(s) \leq m_2(s)$.
4. $|m| = \sum_{s \in S} m(s)$.
5. $\forall s \in S : (\bigcup_{i \in \mathbb{N}} m_i)(s) = \sum_{i \in \mathbb{N}} m_i(s)$
6. $\forall s \in S : (m_1 \setminus m_2)(s) = \max(0, m_1(s) - m_2(s))$

The set of all bags over S is denoted by S_{MS} and the bag containing no elements is denoted by $\emptyset_{\text{MS}}$.

In this thesis, definitions of mathematical formalisms are given. In these formalisms, data expressions are modeled. Thus, we assume that the tools in

which the mathematical formalisms are modeled, provide an expression language. We denote by EXPR the set of expressions provided by the expression language, and by $\text{Type}[e]$ we denote the type of an expression $e \in \text{EXPR}$. In some cases, evaluating an expression e results in multiple values of type C in a bag, the type of such an expression is denoted by $\text{Type}[e] = C_{\text{MS}}$. We denote the set of free variables in an expression $e \in \text{EXPR}$ by $\text{VAR}[e]$. Let V be a set of variables. The set of expressions $e \in \text{EXPR}$ such that $\text{VAR}[e] \subseteq V$ is denoted by EXPR_V .

Definition 2 (Binding). A **binding** of a set of variables V is a function b that maps each variable $v \in V$ to a concrete value $b(v)$ with type $\text{Type}[v]$. The set of all bindings for a set of variable V is denoted by $B[V]$.

Using a binding, the value of an expression with free variables can be evaluated. This is done by replacing the free variables with the value assigned by the binding and evaluating the expression with the concrete values. Evaluating an expression e with binding b is denoted by $e\langle b \rangle$.

Example 1. Consider the set of variables $V = \{n, m\}$, with expression $n + 3 \leq m * 5$. With binding $\langle n = 9, m = 2 \rangle$, the expression can be evaluated as follows:

$$\begin{aligned} (n + 3 \leq m * 5)\langle n = 9, m = 2 \rangle &= 9 + 3 \leq 2 * 5 \\ &= \text{False} \end{aligned}$$

A Labeled Transition System (LTS) is a formal model for describing behavior. An LTS is a directed graph, consisting of states and arcs with action names.

Definition 3 (Labeled Transition System (LTS)). A Labeled Transition System (LTS) is a four tuple $(S, L, \rightarrow, s_0)$ where

- S is a possibly infinite set of states,
- L is a possibly infinite set of actions,
- $\rightarrow \subseteq S \times L \times S$ is a transition relation, and
- $s_0 \in S$ is the initial state

We denote $(s, t, s') \in \rightarrow$ as $s \xrightarrow{t} s'$.

Example 2. Consider the LTS representing a simple coffee and tea machine as shown in Figure 2.1, with four states, s_0 , s_1 , s_2 , and s_3 and the labeled transitions *coin*, *coffee*, *tea*, and *dispense*. After inserting a coin, the machine gives the choice between coffee and tea, then the machine dispenses the coffee or tea and goes back to the initial state.

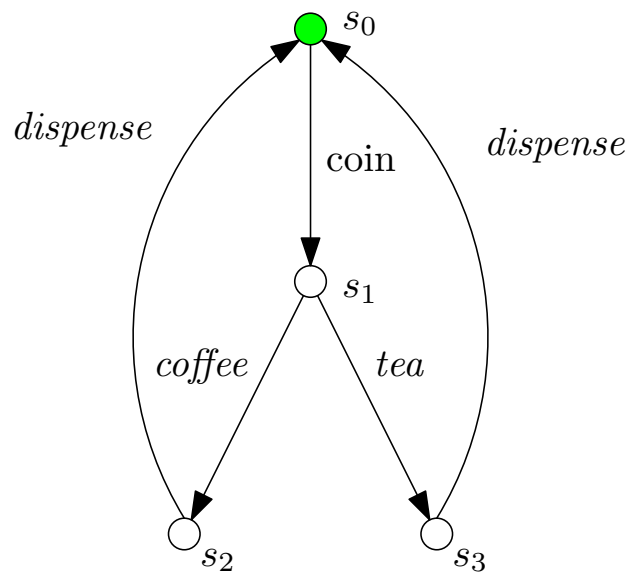


Figure 2.1: An example lts, with the initial state shown in green

Chapter 3

Colored Petri Nets

Colored Petri Nets (CPNs) are an extension of Petri Nets (PN). CPNs can be used to model and specify the behavior of concurrent systems [3]. The semantics of CPNs are well studied and are used in the Matala project to define the semantics of BPMN models. In this chapter, first Petri Nets are defined, and their graphical notation is shown. Then, the Petri Nets are extended to Colored Petri Nets. Lastly, the formal semantics of Colored Petri Nets are explained in terms of Labeled Transition Systems.

3.1 Petri Nets

Petri Nets (PN) [3] is a mathematical formalism for modelling distributed systems, which also supports a graphical notation. A PN is a bipartite directed graph, consisting of places and transitions, together referred to as nodes. Places can be connected with transitions, and transitions can be connected with places using arcs. Petri Nets are abstract models of processes, where work items are processed by a sequence of events. A formal definition is given in Definition 4.

Definition 4 (Petri Net (PN)). *A Petri Net is a triple (P, T, A) , where:*

- *P is a finite set of places,*
- *T is a finite set of transitions ($P \cap T = \emptyset$),*
- *$A \subseteq (P \times T) \cup (T \times P)$ is a set of arcs (flow relation).*

Places are drawn as ovals, transitions are drawn as rectangles, and the flow relation is drawn as arrows connecting places and transitions. Transitions represent the discrete events that take place in a process and tokens represent the work items in the process and are visualized as dots in places. Arrows

model how the tokens move through the model. The graphical representation of Petri Nets is shown in Figure 3.1. The distribution of tokens over the places is called the marking and represents the state of the model. As tokens represent the state, they are not part of the definition.

	Place		Transition
	Place containing a token		Flow relation

Figure 3.1: Graphical representation of PN components

3.2 Colored Petri Nets

Colored Petri Nets extend Petri Nets with typed data. In a CPN, places are extended with a type, called a color, and tokens carry typed data, where the type of a token is the same as the type of the place the token is located in. Transitions can have guards based on the data stored in tokens, and update expressions to manipulate the data. It is assumed that the set of colors of a CPN contains at least the type ‘Bool’, representing the booleans.

Definition 5 (Colored Petri Net (CPN) [3]). *A CPN is a tuple $(P, T, A, \Sigma, V, C, E, G, I)$, where:*

- (P, T, A) is a Petri Net,
- Σ is a finite set of colors.
- V is a finite set of typed variables, such that $Type[v] \in \Sigma$ for all variables $v \in V$.
- $C : P \rightarrow \Sigma$ is a function that assigns a color to each place.
- $G : T \rightarrow EXPR_V$ is a guard function that assigns a guard to each transition t such that $Type[G(t)] = Bool$.
- $E : A \rightarrow EXPR_V$ is an arc expression function that assigns an expression to each arc a such that $Type[E(a)] = C(p)_{MS}$, where p is the place connected to the arc a .
- $I : P \rightarrow EXPR_{\emptyset}$ is an initialization function that assigns an initialization expression to each place p such that $Type[I(p)] = C(p)_{MS}$.

Note that the type of an arc expression e is equal to the bag of the color of the place p the arc expression is connected to, i.e., $Type[e] = C(p)_{MS}$. This means that evaluating an arc expression can result in a bag of values, where the type of all those values is equal to the color of the place the arc is connected to.

The graphical notation of Colored Petri Nets is the same as the notation for Petri Nets. With the colors, guards, expressions, and initialization function shown as annotations in the model, as shown in Figure 3.2.

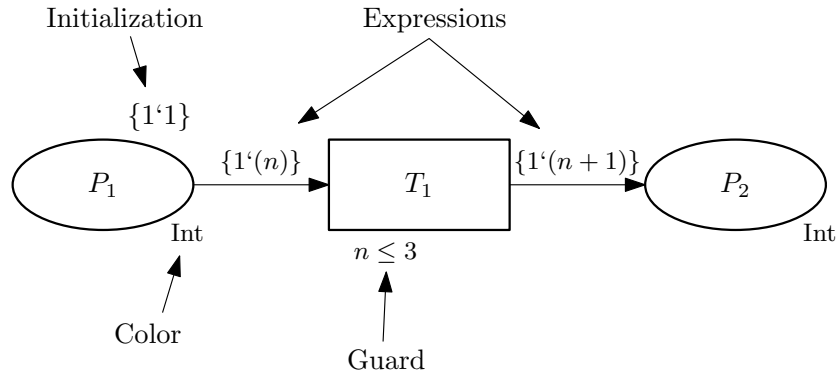


Figure 3.2: Graphical representation of a Colored Petri Net

Example 3. Consider the following CPN model as shown in Figure 3.3. In the initial marking place P_1 contains a token with value 1. Transition T_1 consumes a token from place P_1 and produces a token in P_2 with the value of the token consumed from P_1 incremented by one. If the value of the token in P_2 is less than or equal to three, T_2 consumes a token from P_2 and produces a token in P_1 with the same value. If the value of the token in P_2 is greater than three, T_3 consumes a token from P_2 and produces a token in P_3 with the same value.

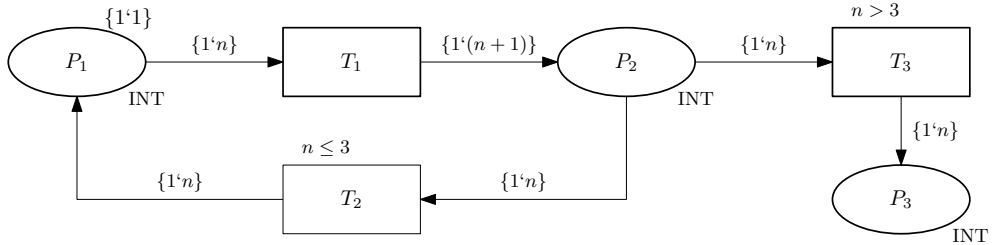


Figure 3.3: Example CPN

3.3 Semantics

For a Colored Petri Net $CPN = (P, T, A, \Sigma, V, C, E, G, I)$, we define some concepts. The **preset** of a transition t , denoted by $pre(t)$, is the set of all places that have an outgoing arc to t , thus, $pre(t) = \{p \in P \mid (p, t) \in A\}$. The **postset** of a transition t , denoted by $post(t)$, is the set of all places that have an incoming arc from t , thus, $post(t) = \{p \in P \mid (t, p) \in A\}$. Similarly as for a PN, the distribution of tokens over the places is called the **marking** and represents the state of the model. Formally, the marking is a function M that maps each place $p \in P$ to a bag of tokens such that $M(p) \in C(p)_{MS}$. The **initial marking** M_0 is defined by $M_0(p) = I(p)\langle \rangle$ for all $p \in P$.

The execution of a CPN is based on the occurring of transitions. Informally, a transition is enabled if each of the places in the preset of the transition contain tokens, and with the data in the tokens the guard of the transition is satisfied. If a transition is enabled, it can occur and consume tokens from the places in the preset of the transition and produce new tokens in the places in the postset of the transition, according to the arc expressions. Arc expressions and guards consist of free variables and in order to assign values to these free variables, bindings are used.

The **variables of a transition** $t \in T$ are denoted by $Var(t) \subseteq V$ where:

$$Var(t) = VAR[G(t)] \cup \bigcup_{p \in pre(t)} VAR[E(p, t)] \cup \bigcup_{p \in post(t)} VAR[E(t, p)]$$

and consist of the free variables appearing in the guard of t and in the arc expressions of the arcs connected to t . A **binding** of a transition $t \in T$ is a binding for all free variables in t . The set of all bindings for a transition t is denoted by the shorthand $B(t) = B[Var(t)]$. A **binding element** is a pair (t, b) such that $t \in T$ and $b \in B(t)$.

Definition 6 (Enabled binding element). *A binding element (t, b) is **enabled** in marking M , if and only if the following two properties are satisfied:*

1. $\forall p \in pre(t) : E(p, t)\langle b \rangle \subseteq M(p)$.
2. $G(t)\langle b \rangle = True$

Definition 7 (Occurring binding element). *When binding element (t, b) is enabled in marking M , it may **occur**, leading to marking M' , defined by:*

$$M'(p) = (M(p) \setminus E(p, t)\langle b \rangle) \cup E(t, p)\langle b \rangle \text{ for all } p \in P$$

Example 4. *Consider the CPN as given in Example 3. The free variables in t_1 are defined as $Var(t_1) = \{n\}$, thus for t_1 , $b = \langle n = 1 \rangle$ would be a valid*

binding. To evaluate the expression $G(t_1) = \text{True}$ with binding $\langle n = 1 \rangle$, write $G(t_1)\langle n = 1 \rangle$, all the free variables in $G(t_1)$ are replaced with the concrete values defined in the binding. Thus:

$$\begin{aligned} G(t_1)\langle n = 1 \rangle &= \text{True}\langle n = 1 \rangle \\ &= \text{True} \end{aligned}$$

Similarly, expression $E(p_1, t_1) = \{1'n\}$ can be evaluated with binding $\langle n = 1 \rangle$ as follows:

$$\begin{aligned} E(p_1, t_1)\langle n = 1 \rangle &= (\{1'n\})\langle n = 1 \rangle \\ &= \{1'1\} \end{aligned}$$

Finally, expression $E(t_1, p_2) = \{1'(n + 1)\}$ can be evaluated with binding $\langle n = 1 \rangle$ as follows:

$$\begin{aligned} E(t_1, p_2)\langle n = 1 \rangle &= (\{1'(n + 1)\})\langle n = 1 \rangle \\ &= \{1'(1 + 1)\} \\ &= \{1'2\} \end{aligned}$$

Thus, for binding b , $G(t_1)\langle b \rangle = \text{True}$ and $E(p_1, t_1)\langle b \rangle \subseteq M(p_1)$, so it is enabled and can occur. After it has occurred the new marking for p_1 would be:

$$M(p_1) \setminus E(p_1, t_1) = \{1'1\} \setminus \{1'1\} = \emptyset_{MS}.$$

And the new marking for p_2 becomes:

$$M(p_2) \cup E(t_1, p_2) = \emptyset_{MS} \cup \{1'2\} = \{1'2\}.$$

When a CPN has marking M and the binding element (t, b) occurs, resulting in marking M' , as specified in Definition 7, we say that M' is **directly reachable** from M , and denote this as:

$$M \xrightarrow{(t,b)} M' \quad \text{or} \quad M \twoheadrightarrow M'$$

The latter only indicates that a binding element can occur from marking M to M' .

Definition 8 (Reachability). A marking M' is **reachable** from marking M if there exist zero or more markings M_1 to M_n , such that:

$$M \twoheadrightarrow M_1 \twoheadrightarrow \dots \twoheadrightarrow M_n \twoheadrightarrow M'$$

The set of all reachable markings from M is denoted by $\mathcal{R}(M)$.

From a CPN a reachability graph as an LTS, see Definition 3, can be computed. With the reachability graph the behavior of a CPN can be compared with other modelling formalisms.

Definition 9 (Reachability graph of a CPN). *The reachability graph of a CPN $(P, T, A, \Sigma, V, C, E, G, I)$ with initial marking M_0 , is an LTS $(S, L, \rightarrow, s_0)$ s.t.:*

- $S = M_0 \cup \mathcal{R}(M_0)$
- $L = T$
- $\rightarrow = \left\{ (M, t, M') \in \mathcal{R}(M_0) \times T \times \mathcal{R}(M_0) \mid \exists b \in B(t) : M \xrightarrow{(t,b)} M' \right\}$
- $s_0 = M_0$

Example 5. *Consider the CPN as shown in Example 3, the reachability graph is shown in Figure 3.4.*

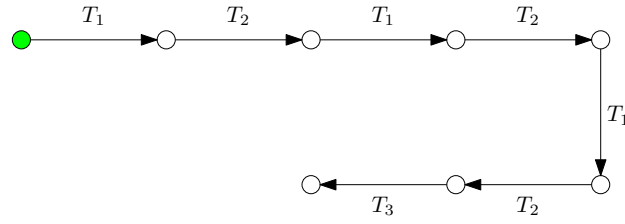


Figure 3.4: The reachability graph of a CPN

Chapter 4

Business Process Modeling & Notation

Business Process Model and Notation (BPMN) is a language to model business processes and collaborations. BPMN is a widely accepted standard for business processes diagrams. It is intended to be used by stakeholders who manage the business processes [2]. Specifically, BPMN is designed to describe distributed, complex scenarios, where multiple actors interact using messages [5].

Due to the use of natural language and ambiguous terms, a drawback of BPMN is the lack of formally defined semantics [5, 6]. This makes it difficult to use a BPMN model as a formal specification of a process. The formalization of BPMN models in terms of Petri Nets and Colored Petri Nets has been extensively studied and nice theoretical results exist about their soundness [7]. Therefore, in the Matala project, the semantics of BPMN models are defined in terms of a mapping to Colored Petri Nets. Using the CPN semantics, a model can be used for Model-based testing and model checking.

In this chapter a definition of BPMN processes and BPMN models is given together with their graphical representation. Then, data specifications in BPMN are explained. An exact translation from BPMN to CPN already exists in the Matala project and is thus not within the scope of this thesis. At the end of this chapter, there is a brief discussion on the mapping from BPMN to Petri Nets. An example of a transition from BPMN to CPN can be seen in Appendix A.

4.1 BPMN syntax

For BPMN we will consider the components as show in Figure 4.1. Each process starts with a **Start location** and ends with an **End location**. **Tasks** represent the steps in the business process modeled by the BPMN model and in the **Data store** typed data can be stored. The **AND** and **OR** gateways model parallel and branching behavior. The processes are contained within a **Swimlane** and can only communicate with processes in other **Swimlanes** using **Data stores** or **Message flows**.

Solid arrows represent the **Control flow** of the model, **Tasks** have exactly one outgoing and exactly one incoming **Control flow** arrow each. A **Start location** has exactly one outgoing **Control flow** arrow. An **End location** has exactly one incoming **Control flow** arrow. A gateway can be a split gateway with one incoming and one or more outgoing control flow arrows, or a join gateway with one or more incoming and one outgoing control flow arrow. Dashed arrows represent the **Data flow** and connect **Data stores** with **Tasks** or **Tasks** from different **Swimlanes** with each other, forming **Message flows**. A formal definition of a BPMN process is given in Definition 10.

	Data store		XOR gate
	AND gate	 	Control flow Data relation
	Task		Start location
	End location		Swim lane

Figure 4.1: Graphical representation of BPMN components

Definition 10 (BPMN Process [7]). *A BPMN process is a tuple $\mathcal{P} = (\mathcal{O}, \mathcal{T}, \mathcal{E}, \mathcal{G}, e^S, e^E, \mathcal{G}^F, \mathcal{G}^J, \mathcal{G}^B, \mathcal{G}^M, \mathcal{F})$ where:*

- $\mathcal{O}$ is a set of objects which is partitioned into disjoint sets of tasks $\mathcal{T}$, events $\mathcal{E}$, and gateways $\mathcal{G}$,

- $\mathcal{E} = \{e^S, e^E\}$ is a set containing the start event e_S and end event e_E ,
- $\mathcal{G}$ is partitioned into disjoint sets of parallel (AND) fork gateways $\mathcal{G}^F$, parallel join gateways $\mathcal{G}^J$, branching (XOR) fork gateways $\mathcal{G}^B$, branching merge gateways $\mathcal{G}^M$, and
- $\mathcal{F} \subseteq \mathcal{O} \times \mathcal{O}$ is the control flow relation.

The transitive closure of $\mathcal{F}$ is denoted by $\mathcal{F}^*$.

Example 6. Example of a BPMN process where first either Task A or Task B occurs and then Task C occurs.

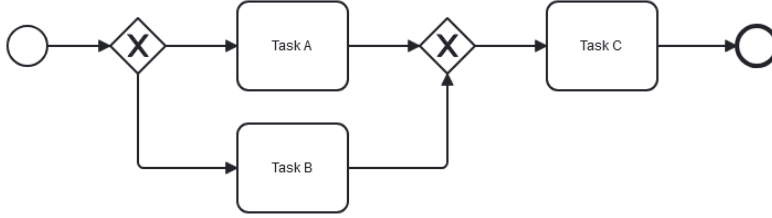


Figure 4.2: Example BPMN process

A BPMN process represents the behavior of a single component. In a BPMN model, multiple components that communicate with each other are modeled. Each component is modeled separately in a swimlane and communication between the swimlanes is modeled as data stores or message flows. Together the swimlanes form the whole BPMN model. To make a distinction between BPMN processes, the elements of a BPMN process $\mathcal{P}$ can be referred to with a subscript $\mathcal{P}$, e.g., the flow relation of $\mathcal{P}$ is denoted by $\mathcal{F}_{\mathcal{P}}$. A formal definition of a BPMN model is given in Definition 11.

Definition 11 (BPMN Model [7]). A BPMN model is a tuple $\mathcal{M} = (\mathcal{Q}, \mathcal{S}, \mathcal{D}, \mathcal{MQ})$ where:

- $\mathcal{Q}$ is a set of BPMN processes,
- $\mathcal{S}$ is a set of data stores, and
- $\mathcal{D} \subseteq \mathcal{S} \times \mathcal{T}_A \cup \mathcal{T}_A \times \mathcal{S}$ is the data relation, where $\mathcal{T}_A = \bigcup_{\mathcal{P} \in \mathcal{Q}} \mathcal{T}_{\mathcal{P}}$, and
- $\mathcal{MQ} = \bigcup_{\mathcal{P}_1 \in \mathcal{Q}, \mathcal{P}_2 \in \mathcal{Q} \setminus \mathcal{P}_1} \mathcal{T}_{\mathcal{P}_1} \times \mathcal{T}_{\mathcal{P}_2}$ are message queues between tasks of different processes.

For any node $x \in \mathcal{O}$, the input nodes of x are given by $in(x) = \{y \in \mathcal{O} \mid y\mathcal{F}x\}$, and the output nodes of x are given by $out(x) = \{y \in \mathcal{O} \mid x\mathcal{F}y\}$.

Based on the definition of BPMN models that is given, models that break the rules that were described informally can be created. For example, it is

possible for a task to be preceded and succeeded by multiple tasks, without a gateway in between, as shown in Figure 4.3. Therefore, it is important to restrict the BPMN models. In this thesis, we will only consider **Well-formed** BPMN models.

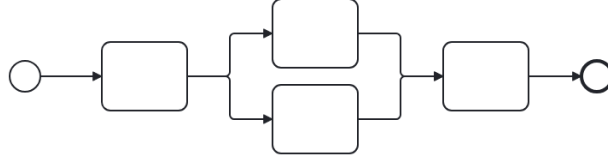


Figure 4.3: A BPMN with an invalid structure

Definition 12 (Well-formed BPMN model). *A BPMN model $\mathcal{M}$ is **Well-formed**, if and only if for each $\mathcal{P} \in \mathcal{Q}$ the following holds:*

- $in(e^S) = \emptyset \wedge |out(e^S)| = 1$, i.e., the start event has an indegree of zero and an outdegree of one,
- $|in(e^E)| = 1 \wedge out(e^E) = \emptyset$, i.e., the end event has an indegree of one and an outdegree of zero,
- $\forall t \in \mathcal{T} : |in(t)| = 1 \wedge |out(t)| = 1$, i.e., all tasks have an indegree of one and an outdegree of one,
- $\forall g \in \mathcal{G}^F \cup \mathcal{G}^B : |in(g)| = 1 \wedge |out(g)| > 1$, i.e., all fork and branch gateways have an indegree of one and an outdegree of greater than one,
- $\forall g \in \mathcal{G}^J \cup \mathcal{G}^M : |in(g)| > 1 \wedge |out(g)| = 1$, i.e., all join and merge gateways have an indegree of greater than one and an outdegree of one,
- $\forall x \in \mathcal{O} : e_S \mathcal{F}^* x \wedge x \mathcal{F}^* e_E$, i.e., every object is on a path from the start event to the end event.

Example 7. Consider the well-formed BPMN model, consisting of two components, as shown in Figure 4.4. The Scheduler component contains a data store with a schedule, one by one the Scheduler dispatches the print jobs from the schedule. After it dispatches a print job, it waits for a response before dispatching the next job. The Printer receives print jobs and subsequently runs the print jobs. After printing it sends a response back to the Scheduler.

4.2 Data in BPMN models

In a BPMN model, data can be modeled as data objects, called tokens, which are present in the model. The tokens can be located in data stores, message queues, and in control flow arrows. The type of a token depends on where it

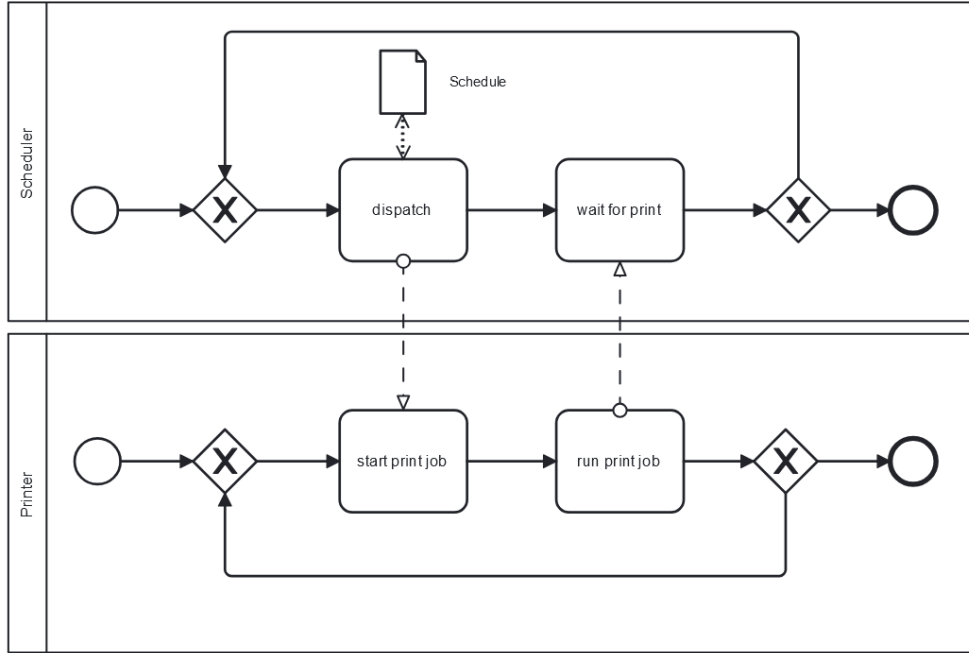


Figure 4.4: Example BPMN model with communication

is located. The type of the tokens in data stores and message queues is the type of the data store or message queue, and type of the tokens in control flow arrows is defined in the swimlane. Tokens are consumed and produced when tasks occur. Tasks can have guards and update expressions, determining when they can occur and the tokens they produce. The semantics of tasks occurring is defined in terms of mapping to Colored Petri Nets.

4.3 A Petri Net semantics of BPMN models

A key idea of BPMN is that models are composed of activity nodes, denoting events or work performed, and control nodes, capturing the flow of control and data between activity nodes. Activity and control nodes can be connected by means of a flow relation [7]. In Petri Nets, some transitions represent the work performed in a process, and a combination of transitions and places capture the control and data flow of the process. Transitions and places are also connected using a flow relation. Thus, the modeling concepts of BPMN and Petri Nets are similar. However, Petri Nets have been well studied as a mathematical formalism, and as discussed in Chapter 3, have well defined formal semantics. Thus, a mapping from BPMN models to Petri Net models give the BPMN models strict semantics.

In this thesis, only non-hierarchical Petri Nets are considered, however the BPMN models consist of multiple swimlanes that each have their own BPMN process and communicate with each other using datastores and queues. In each swimlane there is exactly one start location. However, in a Petri Net, there can be many places from where a process can start. Thus, for the Petri Nets, the swimlanes can be ignored, as show in the translation in Appendix A.

In a Petri Net, transitions can be considered as labeled transitions, or as silent transitions. Labeled transitions represent the work done in a process, similar to tasks in a BPMN process, and silent transitions together with the places capture the control flow that cannot be observed. Tasks in a BPMN model can be modeled as labeled transitions in a Petri Net. Control nodes, such as gateways, can be modeled as silent transitions in combination with places in a Petri Net. Data stores and message queues can be modeled as places.

Example 8. Consider a process with three tasks, *A*, *B*, and *C*, where after task *A* occurs, either task *B* or task *C* can occur. In a BPMN this could be modeled using a branching gateway, as shown in Figure 4.5.

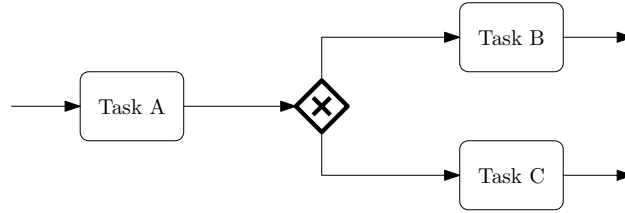


Figure 4.5: BPMN process with a branching gateway

The same process can be modeled as a Petri Net, using a place instead of a gateway, as shown in Figure 4.6.

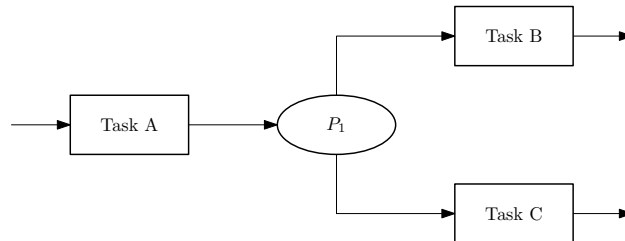


Figure 4.6: Petri Net mode with branching behavior

For a formal definition of a mapping from BPMN to Colored Petri Nets please refer to [7].

Chapter 5

Translating CPNs to mCRL2

The mCRL2 language is a formal specification language built upon the rich literature of process algebras [8]. In this graduation project, the mCRL2 toolset is used as a verification back-end for Colored Petri Nets. This allows for model checking of CPNs, and thus also for BPMN models.

In this chapter, first the definition of the mCRL2 language and a translation from CPNs to mCRL2 is discussed. Then, the semantics of mCRL2 is discussed, and an equivalence between CPNs models and their translation to mCRL2 is established. Finally, some implementation details of the translation are discussed.

5.1 Definitions

To use mCRL2 as a verification back-end, a translation from CPN to a format accepted by the mCRL2 model checker is necessary. The format chosen for this translation is Linear Process Equations (LPEs). An LPE is an mCRL2 process of restricted form. For a process, a set of actions, with possible parameters, *Act* is assumed.

Definition 13 (Linear Process Equation (LPE)[8]). *A Linear Process Equation is a specification of the following shape:*

$$Spec(d : D) = \sum_{k \in K} \sum_{h_k : H_k} c_k(d, h_k) \rightarrow a_k(d, h_k) \cdot Spec(g_k(d, h_k))$$

Where *Spec* is the name of the process,

- *D* is a datatype,
- *d* is a variable of type *D*,

- K is an index set and for all $k \in K$:
 - H_k is a datatype,
 - h_k is a variable of type H_k ,
 - $c_k : D \times H_k \rightarrow \text{Bool}$ is a condition,
 - $a_k \in \text{Act}$ is an action label,
 - $g_k : D \times H_k \rightarrow D$ is the next state.

The notation $\sum_{k \in K} p_k$ is used as a shorthand for $p_{k_1} + \dots + p_{k_n}$ for all $k_1, \dots, k_n \in K$.

5.2 Translating CPN models

To model a CPN using an LPE, the LPE must keep track of the state of the CPN. For a CPN the state is a function mapping places to bags of tokens. In the LPE the state, D , is modeled as an n -tuple of bags of tokens, where n is equal to the number of places in the CPN. Thus in the LPE, similarly as in the CPN, for each place a bag of tokens is stored.

Additionally, the LPE should model the same transitions as the CPN. Since transitions in CPNs do not carry data, the type of the transition and the functions producing the data in the resulting LPE can be omitted. In a CPN, a transition can occur if there exists a binding such that the guard of the transition evaluates to True, and the input places of the transition have tokens, see Definition 6. To model this in the LPE, the guard of a transition $t \in T$, c_t should evaluate to True, if and only if the same conditions as in the CPN hold.

Finally, when a transition occurs in a CPN, the tokens in the input places are removed, and new tokens are placed in the output places, according to the update expressions in the transition, see Definition 7. In the LPE, the same should be modeled in the update function g_t for a transition $t \in T$.

A formal definition of the translation is given below.

Definition 14 (Translation). *Given a CPN $(P, T, A, \Sigma, V, C, E, G, I)$, produce an LPE:*

$$\text{Spec}(d : D) = \sum_{k \in K} \sum_{h_k : H_k} c_k(d, h_k) \rightarrow a_k \cdot \text{Spec}(g_k(d, h_k))$$

Where:

- $K = T$,

- $D = (S_{p_1} \times \dots \times S_{p_n})$, for all $p_1, \dots, p_n \in P$, where for each $p \in P$, S_p is a bag such that $\text{Type}[S_p] = C(p)_{MS}$.

And for all $t \in T$:

- $H_t = (\text{Type}[v_1] \times \dots \times \text{Type}[v_n])$, for all $v_1, \dots, v_n \in \text{Var}(t)$
- $a_t = t$, where a_t has no parameters,

And for $d = (s_{p_1}, \dots, s_{p_n})$ for all $p_1, \dots, p_n \in P$:

- $c_t(d, h_t) = (\forall p \in \text{pre}(t) : E(p, t) \subseteq s_p) \wedge G(t)$,
- $g_t(d, h_t) = ((s_{p_1} \setminus E(p_1, t)) \cup E(t, p_1), \dots, (s_{p_n} \setminus E(p_n, t)) \cup E(t, p_n))$.

For a CPN the initial state is defined as M_0 , therefore the initial state of the LPE, d_0 , is defined for all $p_1, \dots, p_n \in P$ as:

$$d_0 = (M_0(p_1), \dots, M_0(p_n))$$

Example 9. Consider the CPN as defined in Example 3, after translation the resulting LPE becomes:

$$\begin{aligned} \text{Spec}((s_{p_1}, s_{p_2}, s_{p_3}) : (\text{Int}_{MS} \times \text{Int}_{MS} \times \text{Int}_{MS})) = \\ \sum_{h:\text{Int}} (\{1'h\} \subseteq s_{p_1}) \rightarrow t_1 \cdot \text{Spec}(s_{p_1} \setminus \{1'h\}, s_{p_2} \cup \{1'(h+1)\}, s_{p_3}) \\ + \sum_{h:\text{Int}} (\{1'h\} \subseteq s_{p_2} \wedge h \leq 3) \rightarrow t_2 \cdot \text{Spec}(s_{p_1} \cup \{1'h\}, s_{p_2} \setminus \{1'h\}, s_{p_3}) \\ + \sum_{h:\text{Int}} (\{1'h\} \subseteq s_{p_3} \wedge h > 3) \rightarrow t_3 \cdot \text{Spec}(s_{p_1}, s_{p_2} \setminus \{1'h\}, s_{p_3} \cup \{1'h\}) \end{aligned}$$

With initial state $d_0 = (\{1'1\}, \emptyset, \emptyset)$.

5.3 Semantics and equivalence

The semantics of an LPE is an LTS. Thus, to compare the behavior of the LPE with the behavior of the CPN, the LPE is instantiated to an LTS, shown in Definition 15. Then, the instantiation of the LPE can be compared with the reachability graph of the CPN. Similarly as for CPNs, to evaluate expressions variables need to be assigned concrete values. The set of all possible bindings over a datatype H is denoted by $B[H]$.

Definition 15 (Semantics of LPE [9]). *From an LPE resulting from the translation as defined in Definition 14, with a set of actions Act , an LTS with shape $(S, L, \rightarrow, s_0)$ can be induced:*

- $S = D$
- $L = Act$
- $\rightarrow = \{(d_c, a_k, d') \mid d, d' \in D \wedge a_k \in Act$
 $\wedge \exists b \in B[H_k] : c_k(d, h_k)\langle b \rangle = \text{True}$
 $\wedge d' = g_k(d, h_k)\langle b \rangle \wedge d_c = d\langle b \rangle\}$
- $s_0 = d_0$, for a given $d_0 \in D$

To show that the same behavior can be observed from a CPN model as an LPE that is obtained after translating the CPN model as specified in Definition 14, we will prove that the underlying LTSs are bisimilar.

Definition 16 (Bisimulation [8]). *Let $A = (S, L, \rightarrow, s_0)$, and $A' = (S', L', \rightarrow', s'_0)$ be two LTSs. A binary relation $R \subseteq S \times S'$ is called a bisimulation relation if and only if for any two states $s \in S$ and $s' \in S'$ and action a :*

1. *if $s \xrightarrow{a} u$ and sRs' then there exists a state $u' \in S'$ such that $s' \xrightarrow{a} u'$ and uRu' , and*
2. *if $s' \xrightarrow{a} u'$ and sRs' then there exists a state $u \in S$ such that $s \xrightarrow{a} u$ and uRu' .*

Two states s and s' are bisimilar, denoted by $s \Leftrightarrow s'$, if and only if there exists a bisimulation relation R , such that sRs' . Two LTSs are bisimilar if and only if their initial states are bisimilar.

Definition 17. *For a reachability graph $LTS = (S, L, \rightarrow, s_0)$ of a CPN $(P, T, A, \Sigma, V, C, E, G, I)$, and the induced LTS $LTS' = (S', L', \rightarrow', s'_0)$ of an LPE resulting from the translation as defined in Definition 14, introduce a relation $R \subseteq S \times S'$ as follows:*

$$R = \{(M, (M(p_1), \dots, M(p_n))) \mid M \in S\} \quad \text{for all } p_1, \dots, p_n \in P$$

Theorem 1. *For LTSs LTS and LTS' , relation R , as defined in Definition 17, is a bisimulation relation.*

Proof. To show that R is a bisimulation relation for LTS and LTS' , we will show that conditions 1 and 2 from Definition 16 hold.

1. Assume that $(M, (M(p_1), \dots, M(p_n))) \in R$ and that there exists a $t \in L$ such that $M \xrightarrow{t} N$. Since $M \xrightarrow{t} N$, there exists a binding $b \in B(t)$ such that $M \xrightarrow{(t,b)} N$. From that, the following holds:

- $G(t)\langle b \rangle = \text{True}$,
- $\forall p \in \text{post}(t) : E(p, t)\langle b \rangle \subseteq M(p)$, and
- $N(p) = (M(p) \setminus E(p, t)\langle b \rangle) \cup E(t, p)\langle b \rangle$, for all $p \in \text{pre}(t)$.

State $d = (M(p_1), \dots, M(p_n))$ can match transition t with to $(N(p_1), \dots, N(p_n))$ if and only if there exists a binding $b' \in B[H_t]$, such that $c_t(d, h_t)\langle b' \rangle = \text{True}$ and $(N(p_1), \dots, N(p_n)) = g_t(d, h_t)\langle b' \rangle$.

- Since $c_t(d, h_t)\langle b' \rangle = (\forall p \in P : E(p, t)\langle b' \rangle \in M(p)) \wedge G(t)\langle b' \rangle$, for $b' = b$, $c_t(d, h_t)\langle b' \rangle = \text{True}$ holds.
- And:

$$\begin{aligned}
& g_t(d, h_t)\langle b' \rangle \\
&= ((M(p_1) \setminus E(p_1, t)\langle b' \rangle) \cup E(t, p_1)\langle b' \rangle, \\
&\quad \dots, (M(p_n) \setminus E(p_n, t)\langle b' \rangle) \cup E(t, p_n)\langle b' \rangle) \\
&= (N(p_1), \dots, N(p_n)).
\end{aligned}$$

Thus, $(M(p_1), \dots, M(p_n)) \xrightarrow{t} (N(p_1), \dots, N(p_n))$, and $(N, (N(p_1), \dots, N(p_n))) \in R$. So, for all t and all s , for any s' such that sRs' : if $s \xrightarrow{t} u$, then there exists a u' such that $s' \xrightarrow{t} u'$ and $s'Ru'$.

2. Assume that $(M, (M(p_1), \dots, M(p_n))) \in R$ and that there exists a $t \in L$ such that $(M(p_1), \dots, M(p_n)) \xrightarrow{t} (N(p_1), \dots, N(p_n))$. From this it follows that there exists a binding $b \in B[H_t]$ such that:

- $c_t((M(p_1), \dots, M(p_n)), h_t)\langle b \rangle = \text{True}$, and
- $(N(p_1), \dots, N(p_n)) = g_t((M(p_1), \dots, M(p_n)), h_t)\langle b \rangle$

State M can match transition t to N if and only if there exists a binding $b' \in B(t)$ such that $M \xrightarrow{(t, b')} N$. To show that exists a b' such that $M \xrightarrow{(t, b')} N$ the following must hold: $G(t)\langle b' \rangle = \text{True}$, $\forall p \in P : E(p, t)\langle b' \rangle \subseteq M(p)$, and $N(p) = (M(p) \setminus E(p, t)\langle b' \rangle) \cup E(t, p)\langle b' \rangle$ for all $p \in P$.

- Since

$$c_t((M(p_1), \dots, M(p_n)), h_t)\langle b \rangle = (\forall p \in P : E(p, t)\langle b \rangle \in S_p) \wedge G(t)\langle b \rangle$$

for $b' = b$ both $G(t)\langle b' \rangle = \text{True}$ and $\forall p \in P : E(p, t)\langle b' \rangle \subseteq M(p)$ hold.

- Since:

$$(N(p_1), \dots, N(p_n)) = g_t((M(p_1), \dots, M(p_n)), h_t)\langle b \rangle$$

for $b' = b$, for all $p \in P$ $N(p) = (M(p) \setminus E(p, t)\langle b' \rangle) \cup E(t, p)\langle b' \rangle$.

So, $M \xrightarrow{t} N$, and $(N, (N(p_1), \dots, N(p_n))) \in R$. Thus, for all t and all s' , for any s such that sRs' : if $s' \xrightarrow{t} u'$, then there exists a u such that $s \xrightarrow{t} u$ and uRu' .

Since both conditions of Definition 16 hold, R is a bisimulation relation for LTSs LTS and LTS' . $\square$

The initial state of LTS is M_0 , and the initial state of LTS' is d_0 , where for $p_1, \dots, p_n \in P$, $d_0 = (M_0(p_1), \dots, M_0(p_n))$, thus $M_0 R d_0$ and $M_0 \xleftrightarrow{\epsilon} s_0$. Since R is a bisimulation relation for LTS and LTS' , and the initial state of LTS is bisimilar to the initial state of LTS' , $LTS \xleftrightarrow{\epsilon} LTS'$ holds.

5.4 Implementation of the translation

For this thesis, the starting point of the translation is PSpec, a Domain Specific Language (DSL) for describing CPNs [10]. This was chosen because a translation from BPMN to PSpec and a parser for PSpec was already developed. With this parser writing a translation from PSpec to mCRL2 took little effort.

The translation from PSpec to mCRL2 was split up in two steps. First, there is a translation from PSpec to an intermediate CPN format using JSON. Since, the PSpec format preserves the hierarchy that exists in the BPMN models, this translation is used to remove the hierarchy so the CPNs fit Definition 5. Additionally, all expressions are translated to mCRL2 format. Subsequently, the JSON file, together with a separate file containing all type definitions, are translated to LPE format. Finally, the file in LPE format can be parsed to a LPE in the internal format of mCRL2 using the `txt2lps` tool [11, 12]. Alternatively, given that the LPE format is a restricted form of the mCRL2 format, the LPE can be stored as an mCRL2 file, and then translated to the internal LPE format using the `mcr122lps` tool [13]. This is necessary if additional actions are added to the linear equations, as discussed in Chapters 6 and 7.

The translation was split for two main reasons. First, the already existing parser was written using Xtext, a framework for creating DSLs. Using Xtext further would slow down development due to a high learning curve. The second translation is written in Python, which leads to more rapid development. Additionally, for some properties, as discussed in Chapter 6, the model must be modified slightly. These changes serve to expose data or make counterexamples more expressive by showing the value of the tokens. The first translation, from PSpec to the intermediate format, only has one imple-

mentation, and only serves to flatten the CPN and translate the expressions. The second translation is configurable, and thus allows for changes in the resulting specification.

5.4.1 Markings

In a LPE resulting from the translation as defined in Definition 14, tokens are stored in bags. Where for each place in the original CPN, there is a corresponding bag of tokens. Although there are native implementations for bags in mCRL2, using bags to store tokens does not lead to the best performance. Replacing bags with a different data structure can improve the performance. A good alternative to bags are list. With lists, the necessary operations of bags, such as inserting or removing tokens, and a check if tokens are present, can be done much faster.

Alternatively, it is possible that during the execution of a model, the number tokens in a specific place never exceeds one. In that case at most one token needs to be stored for that place. Replacing the marking of a place from a bag or list to a single token can significantly reduce the time it takes to verify a property. However, it must be known beforehand that the tokens in that place never exceeds one. This can either be determined by the modeler, which might lead to mistakes, or by exploring the state space, which might take too much time in comparison with the performance boost.

To show the performance benefits of using lists to represent bags, a performance comparison is made using the CPN as shown in Figure 5.1. In the CPN, place P_1 starts with two hundred distinct tokens, and moves them one by one to P_3 .

To compare the performance of mCRL2, Property 4, as explained in Chapter 6, stating that the transition t_2 should occur exactly two hundred times is checked. Three different implementations are used, where the marking is stored using the build in types Bag, FBag, and List.

Using the mCRL2 tool as explained in Chapter 6, the property is checked ten times for each of the implementations. On average the implementation using the type Bag took 67 seconds and the implementation using FBag took 60 seconds to evaluate the formula, while the implementation using lists only took nine seconds on average. Thus, for this specific example, the implementation using lists was more than seven times faster.

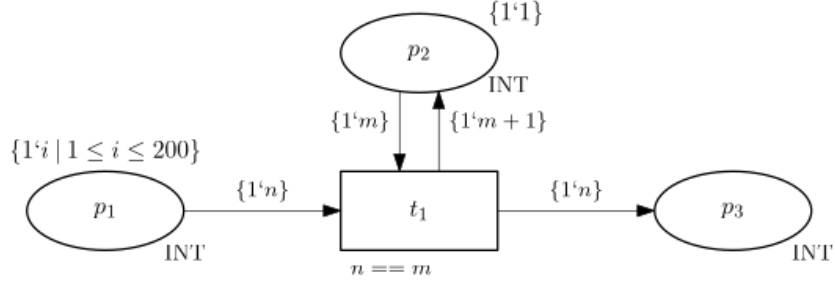


Figure 5.1: CPN for performance comparison

5.4.2 Strings

In mCRL2 there is no native implementation of strings, however the BPMN language does allow for strings. Usually, strings in the BPMN models are added because they are used in the tests generated from the models, but they are not part of the control flow. If, when translating a BPMN model containing strings to an mCRL2 specification, the strings are part of the control flow, strings need to be adapted. If strings are not part of any guard or property, then they do not affect model checking and can thus be ignored. The simplest way of ignoring the strings is by introducing a new datatype called *Unit*, with a single constructor *unit*, and replacing every string with the constructor *unit*.

Chapter 6

Properties for Model Checking

The goal of translating a BPMN model to a CPN, and then subsequently translating the CPN to a LPE is to be able to check if the BPMN model behaves as the modeler intended. However, writing properties in a formal language can be challenging. Using a structured natural language for properties can provide a trade-off between ambiguous properties written in natural language unintuitive properties written in formal language [14].

In this chapter, first a brief explanation of the modal μ -calculus, as used with mCRL2, and how the mCRL2 toolset [15] can be used, is given. Then, some related work is discussed, and finally the property templates are discussed at the end of the chapter.

6.1 Model checking

With mCRL2, the behavior of an LPE can be checked using formal properties specified in the modal μ -calculus, a language for defining properties over an LTS [8]. In this graduation project, only a subset of this language, as shown in Definition 18, is considered.

Definition 18 (the modal μ -calculus). *Let $(S, L, \rightarrow, s_0)$ be an LTS, and Var a set of fixpoint variables. The syntax of μ -calculus in its basic form is given as follows:*

$$\phi := false \mid true \mid \neg\phi \mid \phi \vee \phi \mid \phi \wedge \phi \mid \langle a \rangle \phi \mid [a] \phi \mid \mu X. \phi \mid \nu X. \phi \mid X$$

Where $a \in L$ is an action and $X \in Var$ is a fixpoint variable. Let $A \subseteq L$, be

a finite set of actions, we use the following shorthand notation:

$$\begin{aligned}
\langle A \rangle \phi &= \bigvee_{a \in A} \langle a \rangle \phi & [A] \phi &= \bigwedge_{a \in A} [a] \phi \\
\langle A^* \rangle \phi &= \mu X. (\langle A \rangle X \vee \phi) & [A^*] \phi &= \nu X. ([A] X \wedge \phi) \\
\langle \bar{A} \rangle \phi &= \langle L \setminus A \rangle \phi & [\bar{A}] \phi &= [L \setminus A] \phi
\end{aligned}$$

To denote the complete set of actions L , true is written instead of A . To ensure that the fixpoints $\mu X.\phi$ and $\nu X.\phi$ exists, any occurrence of X in ϕ must be preceded by an even number of negations. For a more in-depth explanation of the modal μ -calculus, please refer to [11] or [8].

Example 10. Let $(S, L, \rightarrow, s_0)$ be an LTS, where $L = \{a, b, c\}$, and $\text{Var} = \{X\}$ a set of fixpoint variables. Consider the following examples:

$$\langle a \rangle \langle b \rangle \langle c \rangle \text{true}$$

This property states that, from the initial state, a trace of an a action, followed by a b action, followed by a c action is possible.

$$[\text{true}^*] \langle \text{true} \rangle \text{true}$$

This property states that after every possible trace, an action is possible, i.e., the model has no deadlocks.

$$\mu X. ([\text{true}] X)$$

This property states that there is only a finite number of steps possible from the initial state, i.e., the model eventually terminates.

With the mCRL2 tool `lps2pbes` [16], from a LPE and a μ -calculus property a Parameterized Boolean Equation System (PBES) [11] can be generated. Subsequently, with the tool `pbessolve` [17], the PBES can be solved to produce a Yes/No answer. This pipeline is shown in Figure 6.1.

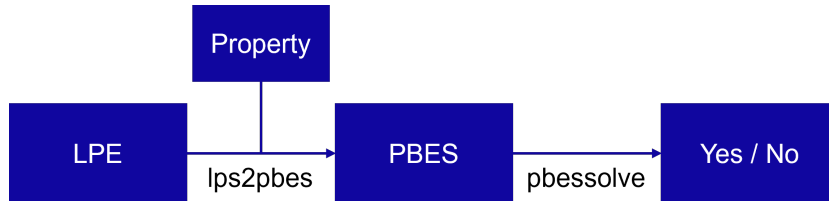


Figure 6.1: Model checking an LPE

To make models, BPMN is provided as a user-friendly interface for CPNs. This lowers the barrier of entry for modelers. A field specialist who creates the BPMN models for Model-Based testing might not be familiar with mCRL2 or the modal μ -calculus. Expecting field specialists to learn the modal μ -calculus and write formal properties in mathematical notation is unrealistic [14]. Thus, a similar interface for the modal μ -calculus would simplify the process of creating properties. As in this scenario, the modeler would not have to interact with mCRL2 or have to write properties in the modal μ -calculus.

6.2 Related work

6.2.1 FRET

The Formal Requirement Elicitation Tool (FRET) [18] is a promising tool for bridging the gap between natural language properties and formal properties. FRET provides a structured natural requirement language, with a mapping to Linear Temporal Logic (LTL) [19, 14].

In FRET, requirements are written in the restricted natural language FRETISH. To aid the user, FRET provides grammar information and English and diagrammatic explanations to clarify the semantics of the temporal properties [18]. Subsequently, the LTL properties can directly be used for other formal methods tools.

A FRETISH requirement is constructed from five fields; the *scope*, *condition*, *component*, *timing*, and *response*. The optional field *scope* specifies that the requirement is only relevant in specific scopes of the system behavior. The optional field *condition* contains a Boolean expression, and specifies that, within the specified scope, the requirement is only relevant when the condition is true. The mandatory field *component* specifies to which component the requirement applies to. The optional field *timing* specifies at which point the response should occur. Finally, the mandatory field *response* specifies the expected response, in the form of a Boolean expression [18].

Given a *scope*, *condition*, *component*, *timing*, and *response*, the requirement is written down as:

<i>scope</i>	<i>condition</i>	<i>component</i>	shall	<i>timing</i>	satisfy	<i>response</i>
--------------	------------------	------------------	--------------	---------------	---------	-----------------

6.2.2 Large Language Models

Like most for problems, Large Language Models (LLMs) have been tried for creating a natural language interface for formal properties. The experimental tool Natural Language to Temporal Logics (NL2TL) [20] has been developed to test translating natural language properties to temporal logics using LLMs.

This is done by first creating a dataset of 28 thousand pairs of natural language with their translation to temporal logic. Then, an existing LLM is finetuned with the dataset. The NL2TL tool can then translate generic natural language properties to Signal Temporal Logic (STL) [20] a superset of Linear Temporal Logic (LTL).

The authors of NL2TL found an accuracy between 90% and 97%, when translating natural language to STL, depending on the specific methodology and LLM used. In practice, this means that one in ten to one in 30 properties can be translated incorrectly. Using an incorrect property can lead to a false negative, i.e., a property does not hold for a model while it should, which leads to strenuous work trying to understand why the property fails. And in the worst case it can lead to a false positive, i.e., a property holds in a model while it should not, which leads to mistakes in the model going undetected.

6.3 Property templates

Specifically looking at the BPMN models that will be developed at ASML, and what kind of properties are important for those models, can allow for a significant reduction in the complexity of the property language. A set of property templates that are specific to the BPMN models that are considered can be developed. To allow for sufficient expressiveness, the templates can be extended with parameters and simple data expressions.

Using these templates, modelers can describe properties in an intuitive way for BPMN models. Given a template, the modeler would only have to complete the template with keywords from the BPMN model, and simple data expressions about the tokens in the model. With the input, the translation of the templates to μ -calculus, and any changes to the model, such as exposing specific data, can be done automatically. Thus, the modeler would not have to write any μ -calculus properties by hand or make any changes to the model.

The selection of property templates was made in consultation with researchers of the Matala project and were designed with possible issues in the BPMN models in mind. The selection was made during the development of

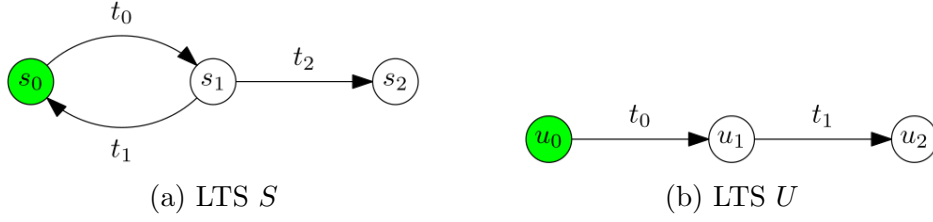


Figure 6.2: The property holds for U but not for S

the MBT methodology in the Matala project.

Some examples of templates, including explanations, are given below. A larger list of templates is given in Appendix B.

Example 11. *Given a BPMN that includes the task ‘dispatch’, the model might want to ensure the task ‘dispatch’ can at most occur 5 times. The following template can be used:*

‘Task A occurs at most n times.’

Where Task A has to be replaced with a task, and n has to be replaced with a number. With the task ‘dispatch’ and 5, the property becomes:

‘dispatch occurs at most 5 times.’

The property can then be translated to μ -calculus.

Some simple properties, like ‘The model should eventually terminate’, do not need any keywords. For these properties, a mapping to a formal definition in μ -calculus is sufficient.

Example 12. *The property: ‘The model should eventually terminate’ can be translated to the μ -calculus formula:*

$$\mu X.([true]X)$$

Formally, this property states that from the initial state, only finite traces should be possible.

Consider the LTSs S and U , as shown in Figure 6.2. From the initial state s_0 in S , an infinite trace of alternating transitions t_0 and t_1 is possible. Thus, the property is violated. From the initial state u_0 in U , all possible traces have a length of at most two. Since the possible traces in U have a finite length, the property holds in U .

6.3.1 Properties regarding the state of the model

Capturing a property that reasons about the state of the model in a μ -calculus formula can lead to convoluted formulae. Consider the property ‘Queue Q

should have at most n items.’ Since the property cannot reason about the state of the LPE, the property would need to keep track of all tasks that are able to put a token in the queue. Writing translations for such properties, and checking such properties is unnecessarily complex.

Instead, it is possible to add an action ‘*raise_exception*’ to the LPE, with the condition that it can only trigger if there are more than n items in queue Q . A queue in BPMN is modeled as a place in a CPN, thus in the mCRL2 specification that results from the translation as defined in Definition 14, there is a state variable S_Q storing a bag of tokens that represents the marking of queue Q . The property $[true^*][raise_exception]false$ verifies that the queue length is never exceeded, and thus verifies the property.

Example 13. Consider the property ‘Queue Q should have at most n items’, and the LPE $Spec(\dots, S_Q, \dots)$ where S_Q is a bag containing the data that represent tokens stored in Q . Assuming the specification does not include the action *raise_exception*, add the following equation to $Spec$, where $\#$ is a function returning the size of a bag:

$$\#S_Q > n \rightarrow raise_exception . Spec()$$

To verify that it is not possible for *raise_exception* to occur, check the altered specification with the property $[true^*][raise_exception]false$.

6.3.2 Properties with data specifications

In a BPMN model, tokens represent the control flow and data flow in the model. In the LPE that results from translating the BPMN model, the tokens are still present. However, they are not exposed, as this would complicate model checking, as discussed in Chapter 5.

Sometimes, however, properties are dependent on the data captured in the tokens. Consider the property ‘If a job with type *TYPEX* is dispatched, *compose_X* should occur before the job is processed’. To check the property, the type of the job has to be exposed. Exposing only the type of the job, instead of the whole token, simplifies checking the property. To expose the correct data of the token, some data expressions need to be written. Additionally, there needs to be some notion of (in)equality to compare the exposed data with the data in the property.

Example 14. Consider the property ‘If dispatch creates an item in the request queue, with type *TYPEX*, *compose_X* should occur before the job is processed’, and an LPE. To expose the type of the job, first the action *dispatch* has to be given the type *JobType*. Then, the type of the item in the request

queue has to be computed. This value is stored in the token produced by the expression $E(\text{dispatch}, \text{request})$, the $TYPEX$ stored in this token is then given as an argument for the action dispatch . Finally, the altered specification is checked with the following property:

$$[true^*][\text{dispatch}(TYPEX)]\mu X.(\overline{[\text{compose_}X]}X \wedge [\text{job_processed}]false \wedge \langle true \rangle true)$$

The property templates provide an intuitive interface for formal properties; however they do limit the modeler. If the modeler would like to express a property that is not an existing template, additional templates have to be made by someone with knowledge of mCRL2 and the modal μ -calculus. This might result in delays. To mitigate this, effort has to be spent ensuring the property templates are sufficiently expressive, and that extending the templates is as simple as possible.

Chapter 7

Results of Model Checking

7.1 Counterexamples

Besides verifying whether a property holds, model checking can also provide evidence to show why a property does or does not hold. If a property does not hold, model checking can provide a counterexample showing how the property is violated. If a property does hold, model checking can provide a witness showing why the property holds. This provides the user with a lot more insight and makes fixing the mistakes in the model simpler. For instance, when a large model is checked, only stating that there exists a deadlock in the model is valuable information. However, it is not feasible to find the deadlock if there are thousands of states. A trace showing how the deadlock occurs is much more useful.

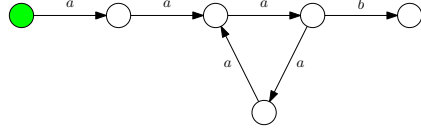
In mCRL2, a counterexample is a sub model of the model given as input to the model checker. Depending on the property, different types of counterexamples can be produced. For properties that state that something bad should not happen, safety properties, a counterexample is a trace leading up to the bad thing happening.

Example 15. *An example of a safety property is a property that states the absence of deadlocks.*

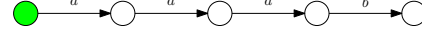
$$[true^*]\langle true \rangle true$$

If this property is violated, the counterexample would be a trace leading up to the deadlock. Consider the model in Figure 7.1a, if the model is checked for deadlock freedom, the trace in Figure 7.1b is produced, showing how a deadlock can be reached.

For properties that state that something good should eventually happen, liveness properties, the smallest counterexample is either a trace leading up

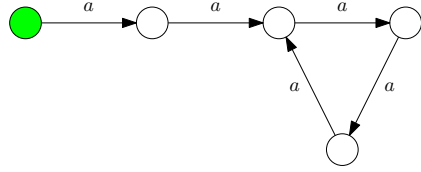


(a) A simple model

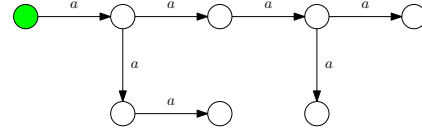


(b) A trace showing the presence of a deadlock

Figure 7.1: Safety properties produce traces as counterexamples



(a) A lasso



(b) A branching model

Figure 7.2: Liveness properties produce lassos or the whole model as counterexamples

to looping behavior, a lasso, indicating the model can get stuck in a loop and it is possible for the good thing to never happen. Or, if the entire model is explored, and the good thing never happens, the counterexample would be a trace to a deadlock. For properties stating that there should exist a future where something good happens, a counterexample could be the whole model.

Example 16. *An example of a liveness property is a property that states eventually a b action should occur.*

$$\mu X.([\bar{b}]X \wedge \langle true \rangle true)$$

This property does not hold for the model shown in Figure 7.1a, the counterexample would be a lasso, as show in Figure 7.2a.

In the model in Figure 7.2b, the action b does not occur. For the property that states that there should exists a trace leading up to a b action:

$$\langle true^* \rangle \langle b \rangle true$$

The counterexample for this property would be the whole model.

7.1.1 Showing tokens in counterexamples

In the LPE resulting from translating a CPN to LPEs, as defined in Definition 14, tokens are not exposed. As the tokens give a lot of information about the

state of the model, it would be clearer if in the counterexamples the token flow was visible.

A solution would be to introduce two new actions *in* and *out*, with both a bag of tokens as datatype. Then, in Definition 14, extend the action a_k to $in(f_k(d, h_k)) \cdot out(j_k(d, h_k)) \cdot a_k$, where

- $f_k(d, h_k) = \bigcup_{p \in P} E(p, k)$, i.e all tokens consumed by transition k , and
- $j_k(d, h_k) = \bigcup_{p \in P} E(k, p)$, i.e all tokens produced by transition k .

To ensure the specification has the correct structure, it should first be linearized using the mCRL2 tool `mcr1221ps` [13]. Now, all the tokens that are produced and consumed by a transition are exposed. However, as the properties only consider actions that are translated from the transitions in the BPMN models, the properties do not need to be changed.

7.2 Message Sequence Charts

Showing only a trace of tasks in the BPMN model to a user can already provide a lot of insights. However, the BPMN models are hierarchical and consist of different components that communicate with each other. With the traces, this information is lost. Showing the tasks in the context of the components they belong to, and the flow of tokens between components, would provide the user with more complete information.

A well-researched method of showing execution traces with communication between components is Message Sequence Charts (MSC). MSC is a graphical language for describing interactions between system components. Each system component is visualized by a vertical line, where time runs from top to bottom. The exchange of messages is shown as arrows between system components, where the direction of the arrow is the same as the direction of the message [21]. An example MSC with three components and three messages is shown in Figure 7.3.

When translating the traces resulting from model checking to MSC, some considerations have to be made as there are multiple ways of visualizing the same idea. For example, the left to right order of the components in the MSC can have a significant impact on the readability. Additionally, as models, and thus counterexamples, can become large, drawing all components can make it difficult to read the chart.

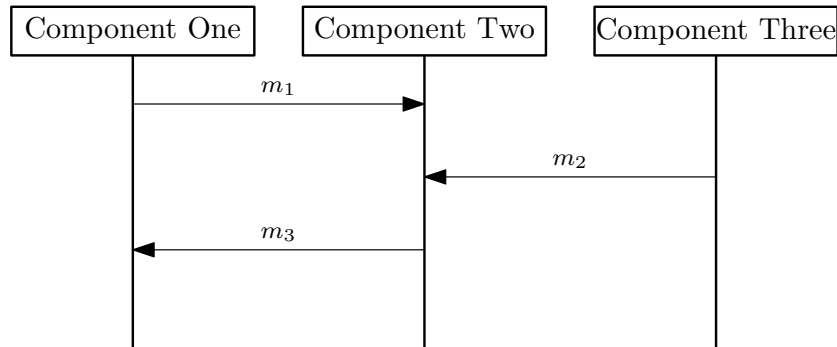


Figure 7.3: Example of a Message Sequence Chart (MSC)

7.2.1 Communication between components

In BPMN, queues and datastores are interfaces between components. All communication between components is modeled as items in queues or datastores. When a component places an item in a queue, it can be retrieved from the queue later on. Thus, communication between components is not atomic, but consist of two separate steps, where other tasks can occur between writing to the queue and reading from the queue.

This can be visualized in multiple ways. To show that communication occurs in two different steps, the queue can be drawn explicitly as a separate actor in the MSC, as shown in Figure 7.4a. Communication between components is shown as multiple communication steps. First, an item is placed in the queue, then it is read by a different component. Reading from the queue is drawn as two arrows to make explicit that the queue is a passive element, and the component initiates the reading.

Alternatively, communication can be drawn as an arrow from the writing component to the eventual recipient, where subsequent reading the message is shown as a box, as shown in Figure 7.4b. This notation is borrowed from Extended Sequence Charts [22]. Lastly, if an item is read from a queue immediately after it is written to the queue, a single arrow that represents the two steps can be sufficient.

The best way to visualize communication between components depends on the situation. If there are multiple components, with multiple queues, drawing all queues explicitly might make an MSC hard to read. Similarly, adding alternative symbols that users might not be familiar with also make the MSCs more complicated.

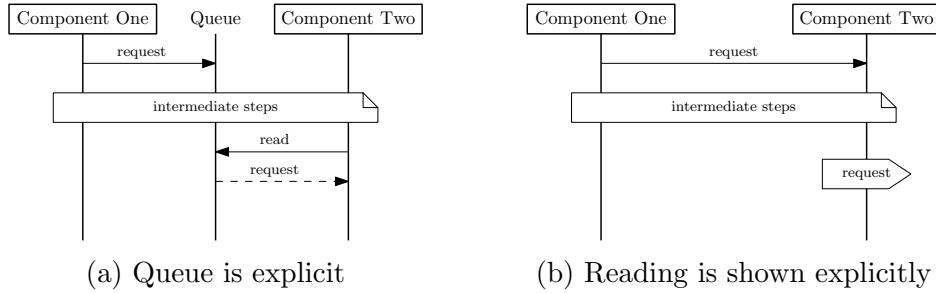


Figure 7.4: Communication between components with queues

7.2.2 Visualizing tasks in Message Sequence Charts

Tasks represent internal behavior of a component and are visualized as boxes over the timeline of a component. Tokens consumed by the task are shown as an arrow from the component where the token originates, to the task. Tokens produced are shown as arrows from the task to the component where the token is later consumed. In Figure 7.5, an MSC with multiple components and tasks is shown.

7.2.3 Visualizing components individually

If a model consists of many components with a lot of communication between them, an MSC that shows all those components can become incomprehensible. Cutting the MSC in smaller parts can prevent this. The components of the BPMN model form natural boundaries for separating the chart in smaller parts. Visualizing a component separately can clearly show the data flow within the component and how the interface is used.

Example 17. Consider the BPMN model as shown in Appendix A, with the property stating there is at most one element in the req queue allowed. If the Schedule datastore contains two schedules, then two items can be placed in the queue, and the property is violated. A MSC of the Scheduler component is shown in Figure 7.6.

For this specific property, the behavior that violates the property occurs only in the Scheduler component. Thus, displaying all components would not have given more information.

7.2.4 Visualizing lassos

A lasso is a trace ending in a loop and can be visualized as a trace where the looping part at the end is shown in a fragment.

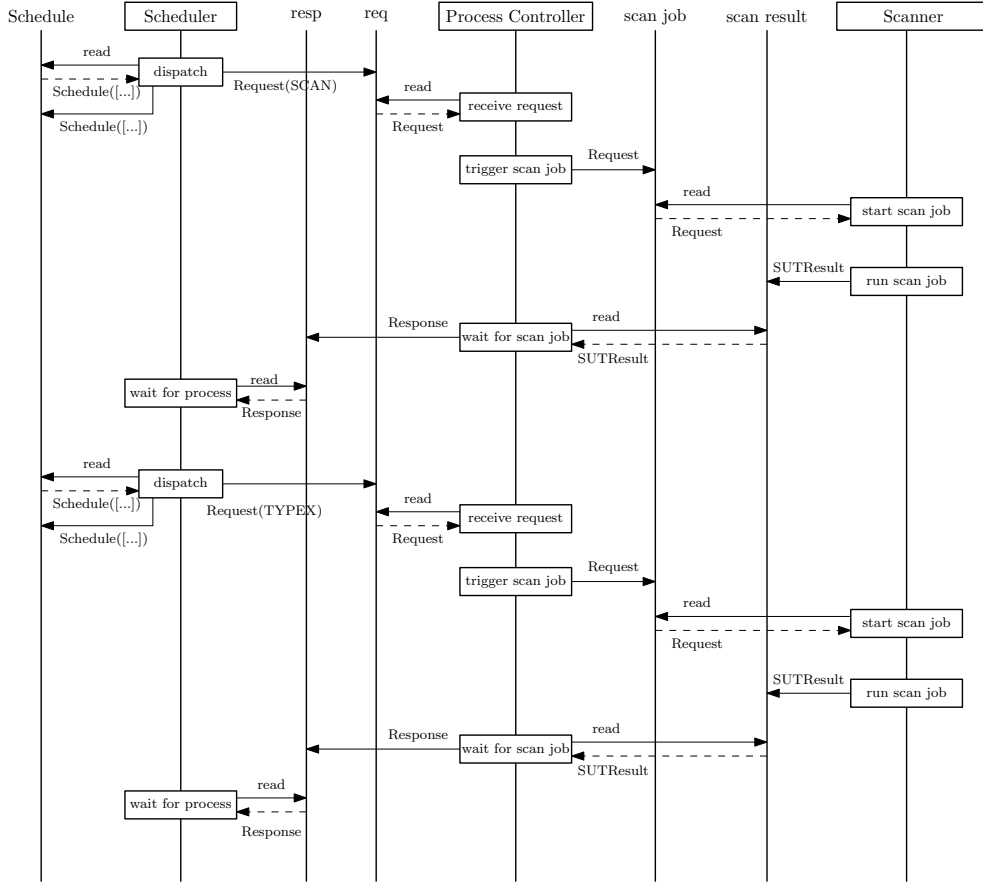


Figure 7.5: MSC with multiple components and tasks

Example 18. Consider the model as shown in Figure 7.2a, where it is assumed transition *A* occurs in Component 1 and sends tokens to Component 2, shown in Figure 7.7.

7.2.5 Limitations

If the BPMN models used for model checking increase in size, the size of the counterexamples can also increase. In Figure 7.5, an MSC with three components and fourteen tasks is shown. To fit this MSC on the page, the size of the font had to be reduced, and there are many vertical lines, this makes it difficult to understand. If a counterexample grows beyond that, it could become even more difficult to understand.

Ideally, an MSC would be generated with a limited the number of components and tasks. The selection could be made automatically, for instance

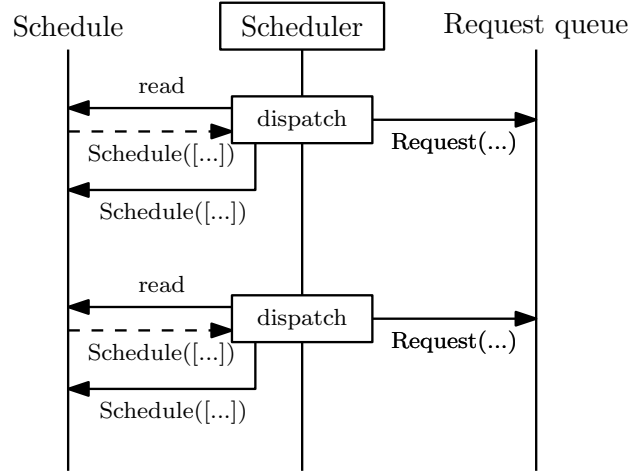


Figure 7.6: Example of an MSC for a single component

by selecting only the last ten tasks that occur, and only generating larger MSCs when requested by the user. Or the selection can be made with input from the user, where the user makes a selection of components and tasks to include before generating the MSC.

Additionally, the property template Property 2, stating that a task is reachable, could lead to counterexample with branching behavior if the entire state space of the model is explored and the property is found not to hold. With an MSC, visualizing such a counterexample with branching behavior is not simple. Showing multiple paths in an MSC is possible, however the notation becomes less clear as the number of branches increases. Alternative paths are shown in a fragment, where the different paths are separated by a dashed line. An alternative that splits up into more branches is shown in a nested fragment, this becomes very messy as the number of branches increases. So, from checking Property 2, visualizing the branching counterexamples does not lead to readable MSCs.

7.2.6 Implementation

An automated translation from counterexamples to MSCs is implemented using SequenceDiagram.org [23], a web-based, textual, and graphical tool for creating sequence diagrams that supports the notation for Message Sequence Charts as discussed in the chapter. Automatic generation of MSC from the counterexamples is difficult as MSCs are very dependent on the user for adjusting the layout. The left to right ordering of the components and queues has a significant impact on the readability. If a component on the

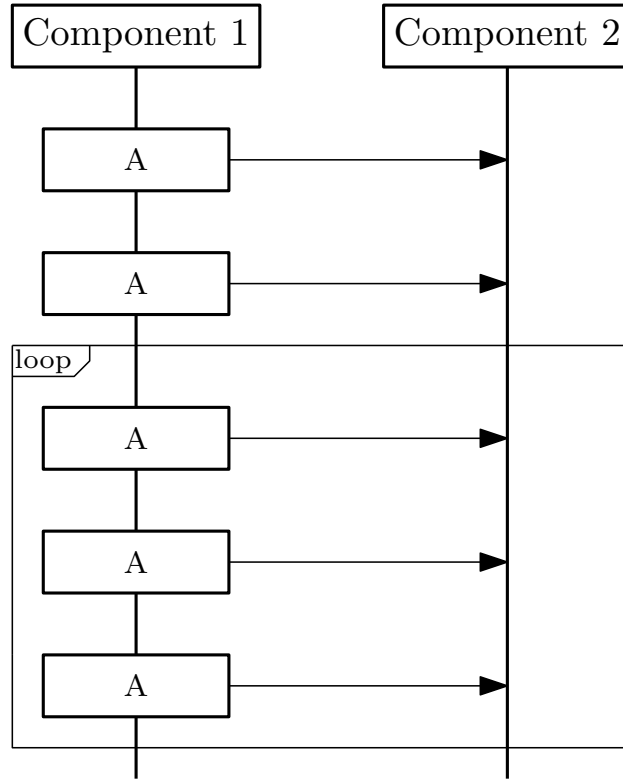


Figure 7.7: A lasso as counterexample

left of the MSC communicates a lot with a queue on the right of the MSC, then a lot of arrows across the entire width of the MSC are needed, this can reduce the readability of the MSC. The naive ordering that is used in the implementation uses the order in which the components and queues occur in the counterexample. Thus, the first occurring component or queue is shown on the left. And the last occurring component or queue is shown on the right.

Additionally, since the hierarchy in BPMN models is removed from the mCRL2 specification, the mCRL2 specification does not store to which components the actions belong. To make this information available for the MSC, the name of the component can be combined with the name of the task, i.e., the task `dispatch` in the component `Scheduler` can be combined to the action `Scheduler_dispatch`. When generating the MSC, the names can be separated again. After the MSC is generated in textual format, SequenceDiagram.org can be used to create the MSC. Using the graphical editor, the order of the components and queues can be improved by the modeler.

7.3 Simulation

During the development of a BPMN model, the first way of checking if a model behaves as expected is by using simulation. Using simulation, a modeler can check if the tokens move through the model as expected. Simple mistakes in the model, such as race conditions that immediately occur, can be discovered. However, as a model grows, the number and length of execution traces increases as well. Since it is not feasible to simulate all feasible execution traces, model checking is used. It would be useful to be able to show a counterexample as a simulation in the modeling tool.

A counterexample consists of the tasks that occurred, the order in which they occur, and for each task which tokens were consumed, and which were produced. Thus, with a counterexample, there is enough information for a simulator to retrace the exact steps with the same data.

Example 19. Consider the BPMN model as shown in Figure 7.8, and the property $\mu X.([\overline{C}]X \wedge \langle true \rangle true)$, expressing that eventually C should occur. If, from the initial state, task A , occurs, then the model is in a deadlock. Thus, the trace containing only task A is a counterexample to the property. A simulation of the trace is shown in Appendix C.

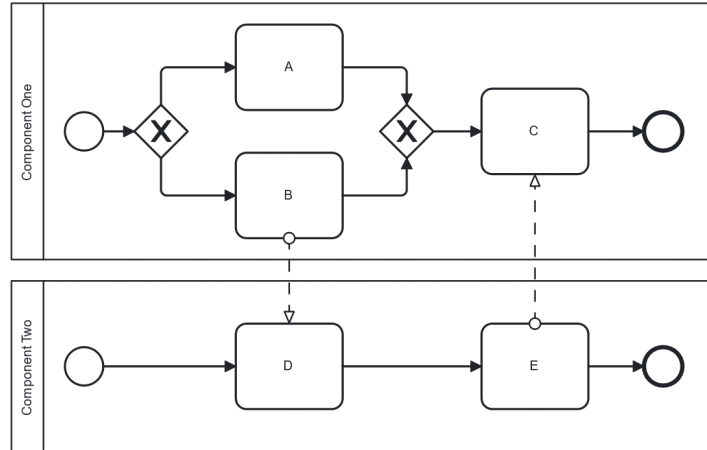


Figure 7.8: Example BPMN model

Since a user has already used the modeling tool, and most likely the simulation tool as well, simulating the counterexample is a very intuitive way of visualizing an execution sequence for the user. This is in contrast to an MSC, where the user would have to get familiar with the notation. However, simulating a counterexample is dependent on the simulation tool being capable of using the output of the model checker.

Implementing simulation of counterexample is at the moment of writing not possible, as the simulation of BPMN models in the tool used to create them is not fully implemented yet. Instead, as a proof of concept, simulations can be made using Microsoft PowerPoint. With an image of the BPMN model, a simulation of tokens can be made using animations.

Chapter 8

Case Study

To validate the method developed in this thesis, a case study is performed. In the case study, a BPMN model, with the translation to mCRL2 is presented for model checking. The model is checked with some properties and the results are shown. Properties that were expected to fail were chosen to emulate the process of having to interpret counterexamples. The case study was presented to the architect of the Matala project to gather feedback.

8.1 BPMN model

For the case study a BPMN model, shown in Figure 8.1, based on four components is used. The specific model is chosen because it is representative of the models the MBT solution is intended for, albeit smaller in size. The model represents two machines, a printer, and a scanner, controlled by a process controller with a schedule. The BPMN model used for MBT can have more process controllers and machines. However, that model was still under development during the thesis.

The schedule component contains a data store with a schedule, and two tasks. The first task takes an item from the schedule and sends it to the process controller. The second task waits until the process controller is finished with the previous item. The process controller receives a request and depending on the request triggers the printer and then afterwards the scanner, or the scanner immediately. After completing the request, the process controller sends back a response to the scheduler signaling it is done. The printer first receives a print job, then, composes a print job of the correct type, and then runs the print job. After running the print job, it sends back a message to the process controller that it is done. The scanner first receives

a scan job, then it runs the scan job. After running the scan job, it sends back a message to the process controller that is done.

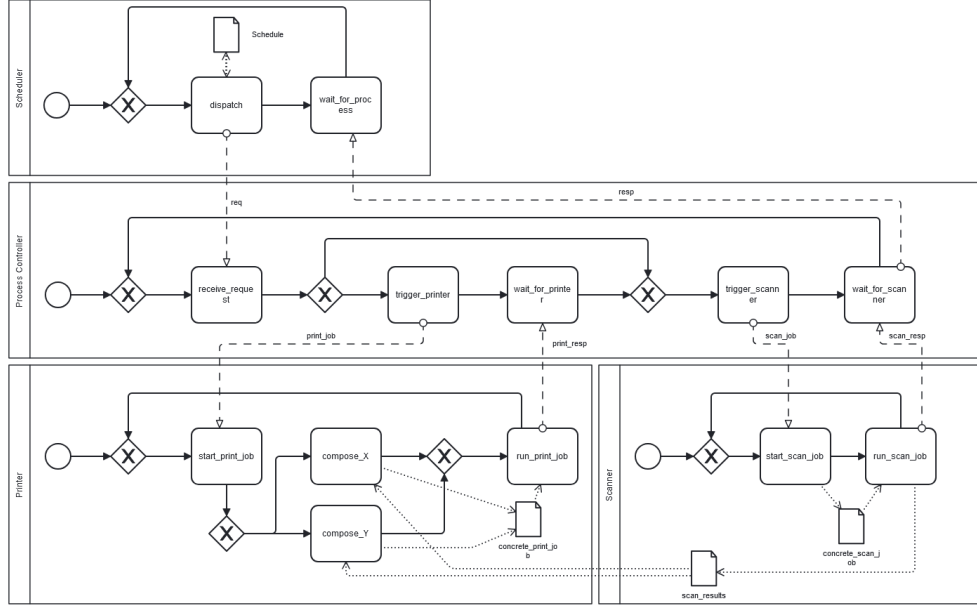


Figure 8.1: BPMN model, see Appendix A for a larger version

The Schedule is initialized with the following list of jobs: **SCAN**, **TYPEX**, **TYPEY**, **TYPEX**, **TYPEY**.

The mapping of the BPMN to a CPN and the subsequent translation of the CPN to LPEs is shown in Appendix A.

8.2 Properties to check

The model consists of a Schedule of items to print and scan. After completing all the items, the model should terminate. Therefore we shall first check if the model terminates, to ensure that the model cannot get stuck in a loop and has no infinite behavior. To check for termination, Property 1 ‘Termination’, as defined in Appendix B, is used. This property holds for the model.

Additionally, we would like to know if any of the guards make it impossible for a task to occur. Therefore we would like to know for each task, whether they are reachable or not. So, Property 2, ‘Reachability’, is checked for each task in the BPMN model. This property also holds for each task in the model.

We would also like to check if the model only terminates after completing

all scheduled tasks. To check this we change the Schedule for this property; we add two lists of jobs, one with four jobs and one with three jobs. To check that all seven jobs are completed we check Property 4, ‘A at least n times’, with the task ‘*wait for process*’ and the number seven. This property also holds for the model.

To ensure the correct job runs, we want to check that a print job with type **TYPEX** results in the task ‘*compose_X*’. To check this we use Property 11, ‘A with $e=c$ ensures B before C’, where Task A is ‘*dispatch*’, e is ‘type’ of the token in queue ‘req’, c is **TYPEX** and Tasks B and C are ‘*Compose_X*’ and ‘*wait for process*’. This property does not hold for the model. To explain why the property does not hold we will visualize it as a Message Sequence Chart, as shown in Figure 8.2. In order to make the MSC less cluttered, components and queues are only shown if they are interacted with.

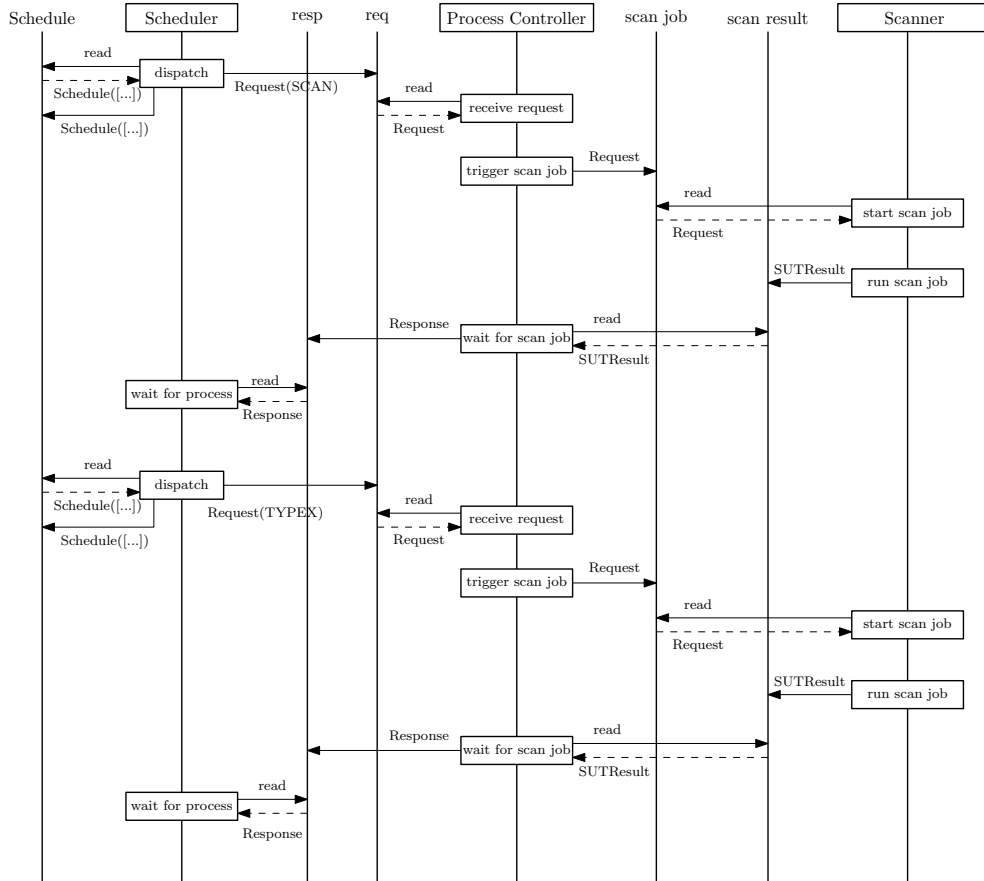


Figure 8.2: Counterexamples for the failing property

In the MSC it is visible that a request with type **TYPEX**, is dispatched.

However, after receiving the request, the ‘Process Controller’ triggers the ‘Scanner’ instead of the ‘Printer’. If we look at the BPMN model, we indeed see that after the task ‘receive request’, there is an XOR gateway, but there is only a guard on the task ‘trigger printer’. Thus, even if the guard is satisfied, it is possible for the model to skip ‘trigger printer’. A possible fix for this issue is to add an additional task ‘skip printer’ after the XOR gateway and add a guard that is satisfied if the type is **SCAN**, and then goes to the XOR join.

This is not the only property template that could have found the mistake. Instead we could have also used Property 3, ‘Eventually’, with the task ‘print job’, or Property 4, ‘A at least n times’, with the task ‘print job’ and the number of times we expect ‘print job’ to occur.

Say we want to change the Schedule. We no longer want to start with a **SCAN** job, but instead skip over it. The new Schedule becomes: **TYPEX**, **TYPEY**, **TYPEX**, **TYPEY**. To see if all jobs are completed, we check Property 4, ‘A at least n times’, with the task ‘*wait for process*’ and the number three. This property now fails as well, the counterexample is shown in Figure 8.3.

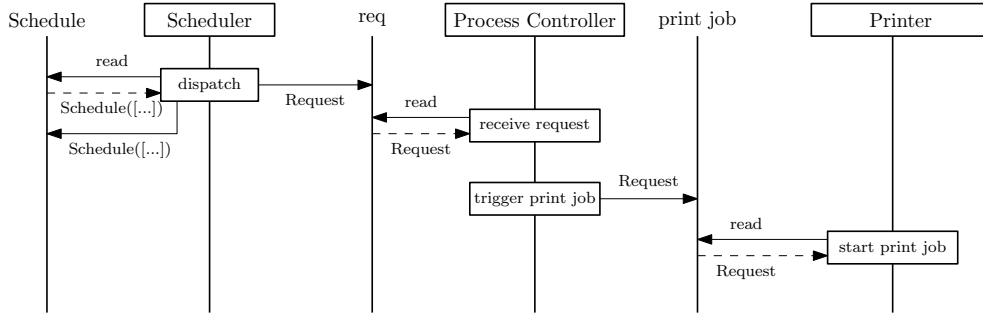


Figure 8.3: Counterexamples for the failing property

The MSC shows a trace ending with ‘*start print job*’, showing that ‘*Compose_X*’ cannot occur. From the MSC, it is not immediately clear why the trace stops after ‘start print job’. However, if the MSC is compared with the BPMN model, it becomes clear that the task ‘*Compose_X*’ needs ‘*scan results*’ as input. Because **SCAN** is removed from the Schedule, the data store ‘scan results’ does not get an item before the printer is triggered. This leads to a deadlock.

8.3 Evaluation

After the case study was completed, it was shown to the architect of the Matala project for feedback. With the feedback, the results of the thesis were evaluated, and where necessary the thesis was improved.

The property templates, as listed in Appendix B, were found to be sufficient to cover the possible question a modeler might have about the models that are intended to be used for MBT. However, it was noted that it would be interesting to be able to combine properties together. Additionally, the description of the templates were too ambiguous, while discussing the templates the opinions on what the templates meant differed, between participants and from the μ -calculus formulae. To improve on this, the descriptions of the template were extended to reduce the ambiguity, and example models that pass and models that fail the properties were added.

From the perspective of a modeler, the Message Sequence Chart were difficult to follow. Even though a modeler might be very familiar with a model, understanding an MSC of a trace of the model takes time. The MSC contains information about the tasks and the flow of tokens, but to understand what is happening in the MSC, the modeler has to, step by step, compare the MSC with the model to understand the trace. However, the aim of the MSCs was to provide a simple diagram of why a property fails. For a modeler, seeing a simulation of why a property fails is much simpler, as the difficult work of retracing the steps is done by the simulator, not by the modeler.

Chapter 9

Conclusion

In this thesis the research question “*How to efficiently and effectively use model checking to analyse BPMN models?*” is discussed. This research question is split in three parts.

The first research question, “*Can a correct and efficient translation be defined from BPMN specifications to mCRL2 specifications to allow for model checking?*”, discusses the translation from BPMN to mCRL2. In the Matala project, the semantics of BPMN is defined as a mapping from BPMN to CPN, and in this thesis a translation from CPN to mCRL2, such that they describe the same behavior, is introduced. Thus, BPMN models can be translated to mCRL2 specifications. In Chapter 5, we proved that the behavior of the CPN models and the mCRL2 specifications is exactly the same, thus properties checked using the mCRL2 specification hold if and only if they hold for the BPMN models.

The second research question, “*Can formal properties be expressed in an easy-to-understand way?*”, discusses designing an easy-to-understand language for stating formal properties. In this thesis, a template language for expressing formal properties is proposed. Although these templates are easy to use, their expressiveness is limited. Extending the set templates to make them more expressive is possible, but labor intensive. Therefore, we cannot conclude that there is an easy-to-understand way of expressing formal properties. In the Matala project, the focus can be limited to only the types of properties that are commonly stated. This makes it easier to create a set of property templates that cover most needs, but there is no guarantee that the type of properties commonly stated remains the same over time.

The third research question, “*Can the results of model checking be presented in an understandable way?*”, discusses presenting the counterexamples

that result from model checking to the users. In this thesis, two methods are proposed showing the counterexamples as Message Sequence Charts (MSCs) and showing the counterexamples as a model simulation. Message Sequence Charts can give great insights in how tokens move through the BPMN model but are only intuitive to follow for users who are familiar with sequence diagram. In practice, using MSCs lead to painstaking work by comparing the MSC with the model. Additionally, automatically generating MSCs from counterexamples is difficult as they can lead to cluttered diagrams. Counterexamples can also be shown as a simulation. Since the simulator is part of the BPMN modelling tool and simulating the model is the first step of verifying if the model behaves as the user intended, the user is most likely already familiar with simulating the model. Thus, showing the exact trace leading up to a mistake in the simulator should give the user a good understanding of why the property has failed. Thus, it can be concluded that simulating the counterexamples in the BPMN modelling tool would be a better alternative way of presenting the results of model checking.

In conclusion, a method for using model checking for BPMN models is laid out in this thesis. Using the sound translation and the property templates the mCRL2 model checker can be used. The results can then be presented to the user. This is only limited by the property templates. In the case study, it was found that the list of property templates was sufficient for the models that the Matala project is intended to be used for. Additionally, it was found that although the MSC were difficult to follow, simulating the counterexamples gave good insights into why a property did not hold in the model. Thus, if model checking is fully integrated in the Matala project, it can be a useful tool for increasing the confidence in the correctness of the models used for Model-Based testing.

Chapter 10

Future Research

After having implemented mCRL2 as a verification back-end, there are several open questions and opportunities. In this chapter, these questions and opportunities are discussed.

10.1 Efficiency of model checking

As discussed in Chapter 9, the usability of model checking greatly depends on the time it takes to verify the model. Therefore it is crucial to keep improving the methodology of model checking BPMN models to reduce the waiting time. There are several open questions in the implementation of the translation proposed in this thesis. Is the current encoding of the marking the most optimal, or are there other data structures that could lead to improvements? Additionally, are there any tools in the mCRL2 toolset or from other toolsets that could improve the performance?

Furthermore, the translation as given in this thesis is not the only possible translation. Exploring different translations lead to improvements. BPMN models consist of components that communicate with each other, whereas the LPE resulting from the translation consists of a single specification. Translating and then model checking the components separately could be faster, as the individual components are much smaller in size than the whole model.

Lastly, with the current implementation, using model checking still takes manual work. From a model, modelers can use a command-line tool to automatically translate a BPMN model to an mCRL2 specification, and a property templates to μ -calculus formulae. Then, the formulae will automatically be checked against the model. For MBT, the goal in Matala project is to provide a single button solution to transform a BPMN model into a test

suite. Similarly, it would be ideal to integrate model checking fully in the modelling tool, where, with a single click, model checking would be done in the background, and after checking all the properties are checked, the results are shown to the modeler.

10.2 Steering test case generation

After a model has been created, it is intended to be use for Model-Based testing. Thus, from the abstract model, test cases will be generated. With MBT the hope is that test coverage is improved by creating more, longer, and more diverse test cases. The number of test cases that can be generated from a model is only limited by the size of the state space of the model. However, if a model grows and concurrent behavior is introduced, the state space of the model, and thus the number of test cases, can grow exponentially. A test suite can grow so large with test cases that are very similar, that is no longer feasible to execute all test. In such cases, to effectively use MBT, a subset containing the most important test cases has to be selected. Picking the most important test cases can be done by hand, or by steering the generation of the test cases. Model checking could be used for steering the test case generation.

As discussed in Chapter 7, if a property holds, a witness showing how the property is satisfied can be generated. A witness has the same structure as a counterexample, a sub model of the model given as input. These sub models can be given as input for test case generation.

Example 20. *Consider a BPMN model with the task ‘handle error’, that should occur if at some point there is an error in one of the components, and a test case where ‘handle error’ occurs has to be generated. If the property stating ‘It should be possible for handle error to occur’, stating that there should be at least one trace in which ‘handle error’ occurs, holds for the model, the witness would be a trace leading up to the task ‘handle error’. Using that trace the requested test case can be generated.*

The witnesses could be found by manually exploring the state space; however the hope is that with model checking this can be done more efficiently. With mCRL2, the exploration of the state space might be more efficient than a custom algorithm. Additionally, using model checking the witness that is produced is based on the property that is specified, whereas a custom algorithm would need to be adjusted for each test case. It would be interesting to research the applications of model checking for steering test case generation, especially how it compares to other ways of steering the generation of test cases.

10.3 Assertions in BPMN models

In this thesis, possible ways of describing properties are discussed. However, writing properties is not the only way of verifying that a model behaves as the modeler intended. An alternative would be to add assertions to the model, and then use mCRL2 to verify whether these assertions always hold in the model. The assertion could be a data expression, which should hold at a time specified by the modeler, e.g., always, like an invariant, or after a specific task occurs.

Example 21. *Consider a BPMN model with a task A . The modeler would like to know if after task A occurs, expression e holds. The modeler would then use the BPMN modelling tool to add expression e as an assertion to be checked after task A occurs. To verify the assertion, the translation from the BPMN model to the mCRL2 specification $Spec(\dots)$ for task A has to be extended from:*

$$c(\dots) \rightarrow A \cdot Spec(\dots)$$

to:

$$c(\dots) \rightarrow A \cdot (e \rightarrow Spec(\dots) \diamond raise_exception)$$

Indicating that after A occurs, if expression e evaluates to `True`, the model should continue as normal, however, if e evaluates to `false`, the model should terminate after `raise_exception`.

Then, the property $[true^][raise_exception]false$ can be used to check if `raise_exception` can occur.*

These assertions are similar as the template discussed in Chapter 6.3.1, but can be extended much further. These assertions seem easy to use, but very limited. As described here, it would be difficult to reason about the order of tasks or about things that should happen in the future. It would be interesting to research the effectiveness of such assertions and if they can be used instead of, or in combination with formal properties.

Bibliography

- [1] J. Tretmans, “On the existence of practical testers,” in *ModelEd, TestEd, TrustEd: Essays Dedicated to Ed Brinksma on the Occasion of His 60th Birthday* (J.-P. Katoen, R. Langerak, and A. Rensink, eds.), pp. 87–106, Springer International Publishing, 2017.
- [2] OMG, “Business process model and notation (BPMN).” <https://www.omg.org/BPMN/>. Accessed: 08-07-2024.
- [3] K. Jensen and L. M. Kristensen, *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Springer Publishing Company, Incorporated, 1st ed., 2009.
- [4] S. Cranen, S. Luttik, and T. Willemse, “Evidence for fixpoint logic,” in *24th EACSL Annual Conference on Computer Science Logic (CSL 2015), September 7-10, 2015, Berlin, Germany*, Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2015.
- [5] F. Corradini, C. Muzi, A. Morichetta, B. Re, and F. Tiezzi, “Well-structuredness, safeness and soundness: A formal classification of BPMN collaborations,” *Journal of Logical and Algebraic Methods in Programming*, vol. 119, p. 100630, 11 2020.
- [6] A. Correia and F. Brito e Abreu, “Adding preciseness to BPMN models,” *Procedia Technology*, vol. 5, p. 407–417, 12 2012.
- [7] R. Dijkman, M. Dumas, and C. Ouyang, *Formal semantics and automated analysis of BPMN process models*. Technical Report Preprint, Queensland University of Technology, 2007.
- [8] J. F. Groote and M. R. Mousavi, *Modeling and Analysis of Communicating Systems*. The MIT Press, 08 2014.
- [9] B. Ploeger, J. Wesselink, and T. Willemse, “Verification of reactive systems via instantiation of parameterised boolean equation systems,” *Information and Computation*, vol. 209, no. 4, pp. 637–663, 2011.

- [10] D. Bera, “GitHub - dbera/OfflineMBT: Systems Modeling and Test Generation — github.com.” <https://github.com/dbera/OfflineMBT>. [Accessed 16-01-2025].
- [11] M. Atif and J. Groote, *Understanding Behaviour of Distributed Systems Using mCRL2*. Studies in Systems, Decision and Control (SSDC), Germany: Springer, 2023.
- [12] “txt2lps; mCRL2 202407.1 documentation — mcrl2.org.” https://mcrl2.org/web/user_manual/tools/release/txt2lps.html. [Accessed 29-01-2025].
- [13] “mcrl22lps; mCRL2 202407.1 documentation — mcrl2.org.” https://mcrl2.org/web/user_manual/tools/release/mcrl22lps.html. [Accessed 29-01-2025].
- [14] D. Giannakopoulou, T. Pressburger, A. Mavridou, and J. Schumann, “Automated formalization of structured natural language requirements,” *Information and Software Technology*, vol. 137, p. 106590, 2021.
- [15] O. Bunte, J. F. Groote, J. J. A. Keiren, M. Laveaux, T. Neele, E. P. de Vink, W. Wesselink, A. Wijs, and T. A. C. Willemse, “The mcrl2 toolset for analysing concurrent systems,” in *Tools and Algorithms for the Construction and Analysis of Systems* (T. Vojnar and L. Zhang, eds.), (Cham), pp. 21–39, Springer International Publishing, 2019.
- [16] “lps2pbcs; mCRL2 202407.1 documentation — mcrl2.org.” https://mcrl2.org/web/user_manual/tools/release/lps2pbcs.html. [Accessed 29-01-2025].
- [17] “pbessolve; mCRL2 202407.1 documentation — mcrl2.org.” https://mcrl2.org/web/user_manual/tools/release/pbessolve.html. [Accessed 29-01-2025].
- [18] D. Giannakopoulou, T. Pressburger, A. Mavridou, J. Schumann, J. Rhein, and N. Shi, “Formal requirements elicitation with fret,” *International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ-2020)*, 2020.
- [19] E. Conrad, L. Titolo, D. Giannakopoulou, T. Pressburger, and A. Dutle, “A compositional proof framework for fretish requirements,” 2022.
- [20] Y. Chen, R. Gandhi, Y. Zhang, and C. Fan, “NL2TL: Transforming natural languages to temporal logics using large language models,” *arXiv preprint arXiv:2305.07766*, 2023.

- [21] M. Reniers, *Message sequence chart : syntax and semantics*. Phd thesis 1 (research tu/e / graduation tu/e), Mathematics and Computer Science, 1999.
- [22] J. Grabowski, P. Graubmann, and E. Rudolph, “The standardization of message sequence charts,” in *Proceedings 1993 Software Engineering Standards Symposium*, pp. 48–63, IEEE, 1993.
- [23] “sequencediagram; SequenceDiagram.org - UML sequence diagram on-line tool.” <https://sequencediagram.org/>. [Accessed 10-03-2025].

Appendix A

Full Translation

In this appendix, a full translation from a BPMN model to a mCRL2 specification is shown in steps. In the first step the BPMN model, as shown in Figure A.1, is translated to a CPN model, as shown in Figure A.2. In the CPN model, the swimlanes from the BPMN model are shown as dashed boxes. A translation of the Scheduler swimlane of the CPN to mCRL2 is shown in Listing A.1

The following datatypes are used in the models, where **CPJ**, **CSJ**, and **SR**, represent abstract values that do not influence the control flow of the model:

```
enum Type { SCAN TYPEX TYPEY }
enum CPJ { cpj }
enum CSJ { csj }
enum SR { sr }

record Schedule {
    int sch_id
    int pr_id
    int sc_id
    Type[] type
}

record Request {
    int sch_id
    int pr_id
    int sc_id
    Type type
}

record PCctx {
    int sch_id
    Request req
}

record PRctx {
    int pr_id
    Request req
}

record SCctx {
    int sc_id
    Request req
}
```

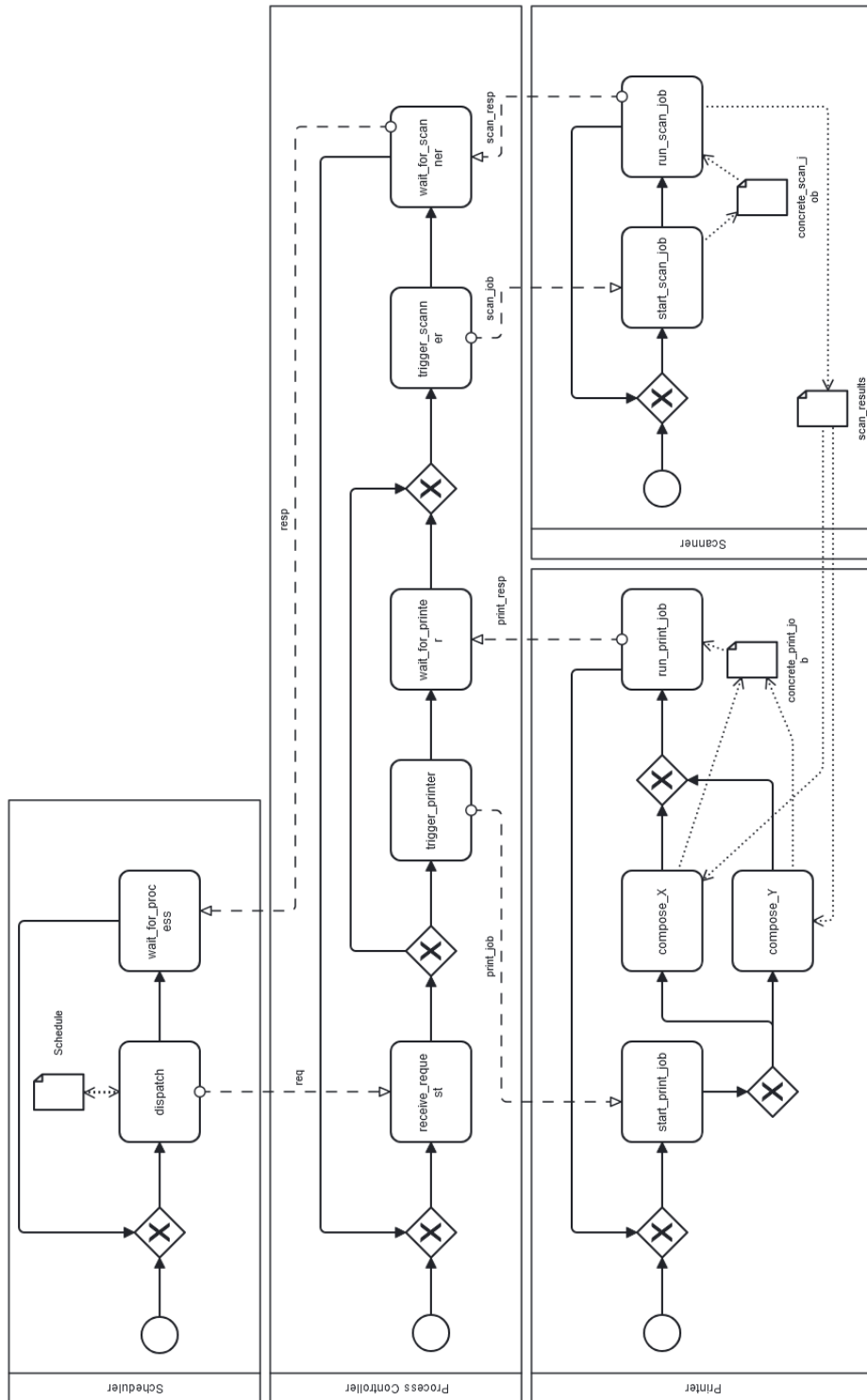



Figure A.1: BPMN model

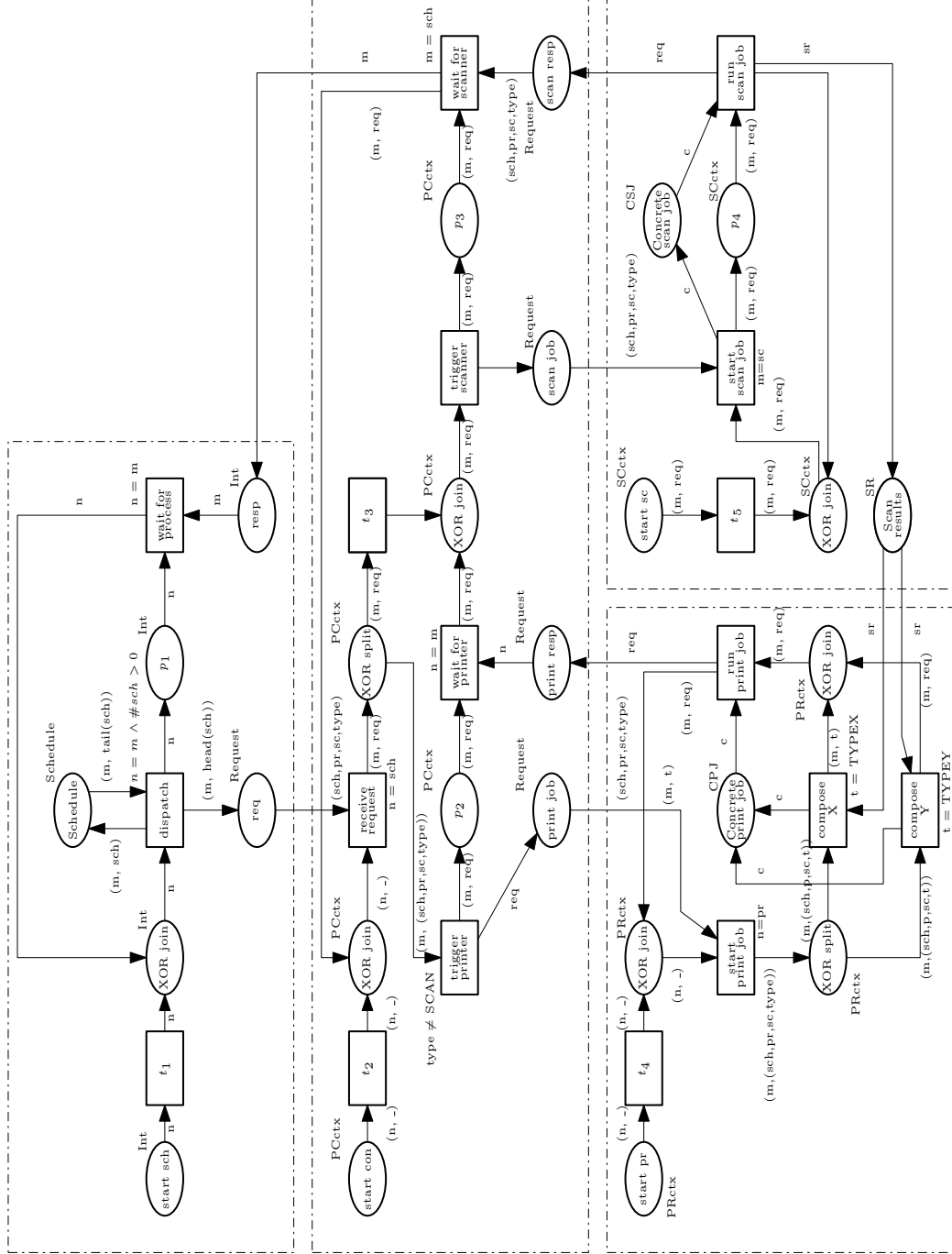


Figure A.2: CPN model, with the swimlanes visualized as dashed boxes

```

sort b.t1 = struct b.t1(start_sch.col: Int);
sort b.dispatch = struct b.dispatch(XOR_join_1.col: Int, schedule.col: Schedule);
sort b.wait_for_process = struct b.wait_for_process(pl.col: Int, resp.col: Int);

act t1, dispatch, wait_for_process;

proc Spec(m.start_sch: List(token),
  m.XOR_join_1: List(token),
  m.pl: List(token),
  m.schedule: List(token),
  m.req: List(token), m.resp: List(token)) =

sum b: b.t1 . (start_sch(start_sch.col(b)) in m.start_sch) ->
  (t1 . Spec(
    m.start_sch = remove_token_from_list(
      start_sch(start_sch.col(b)),
      m.start_sch
    ),
    m.XOR_join_1 = XOR_join_1(start_sch.col(b)) |> m.XOR_join_1
  ) +

sum b: b.dispatch . (
  XOR_join_1(XOR_join_1.col(b)) in m.XOR_join_1
  && schedule(schedule.col(b)) in m.schedule
  && (XOR_join_1.col(b) == sch_id(schedule.col(b))
  && #type(schedule.col(b)) > 0)
) ->
  (dispatch . Spec(
    m.XOR_join_1 = remove_token_from_list(
      XOR_join_1(XOR_join_1.col(b)),
      m.XOR_join_1
    ),
    m.pl = pl(XOR_join_1.col(b)) |> m.pl,
    m.schedule = schedule(Schedule(
      sch_id(schedule.col(b)), pr_id(schedule.col(b)),
      sc_id(schedule.col(b)), tail(type(schedule.col(b)))
    )) |> remove_token_from_list(
      schedule(schedule.col(b)),
      m.schedule
    ),
    m.req = req(Request(
      sch_id(schedule.col(b)), pr_id(schedule.col(b)),
      sc_id(schedule.col(b)), head(type(schedule.col(b)))
    )) |> m.req
  ) +

sum b: b.wait_for_process . (
  pl(pl.col(b)) in m.pl
  && resp(resp.col(b)) in m.resp
  && (pl.col(b) == resp.col(b))
) ->
  (wait_for_process . Spec(
    m.XOR_join_1 = XOR_join_1(pl.col(b)) |> m.XOR_join_1,
    m.pl = remove_token_from_list(pl(pl.col(b)), m.pl),
    m.resp = remove_token_from_list(resp(resp.col(b)), m.resp)
  ));

```

Listing A.1: mCRL2 model of the Scheduler swimlane

Appendix B

Property Templates

Property 1 (Termination). The model terminates. There is only a finite set of steps possible before the model is guaranteed to stop.

Formula:

$$\mu X.([true]X)$$

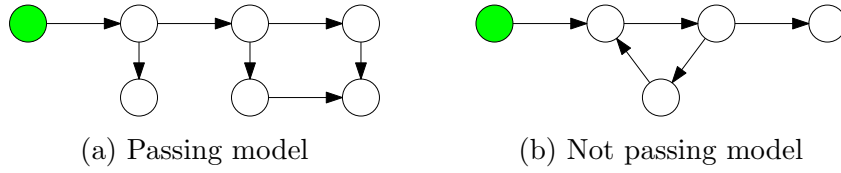


Figure B.1: A passing and not passing model for the property

In the LTS shown in Figure B.1a, only a finite number of steps are possible. However, in the LTS shown in Figure B.1b, the model can get stuck in a loop, and thus an infinite number of steps are possible.

Property 2 (Reachability). Parameters:

- Task A

From the initial state, task A is reachable. From the initial state, there exists a trace leading to Task A .

Formula:

$$\langle true^* \rangle \langle A \rangle true$$

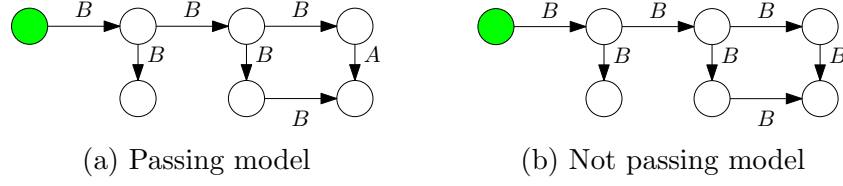


Figure B.2: A passing and not passing model for the property

In the LTS shown in Figure B.2a, there exists a trace, $BBBA$ that ends with A . However, in the LTS shown in Figure B.2b, action A does not occur.

Property 3 (Eventually). Parameters:

- Task A

From the initial state, task A should eventually occur. If the model does not terminate, task A occurs in a finite number of steps, if the model does terminate, task A occurs before the model terminates.

Formula:

$$\mu X.([\bar{A}]X \wedge \langle true \rangle true)$$

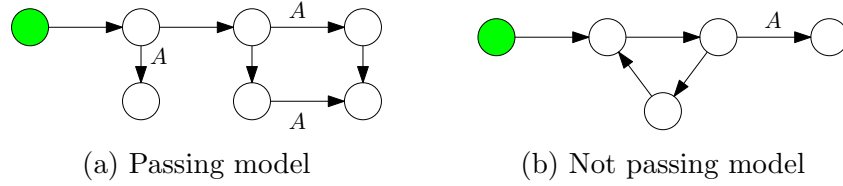


Figure B.3: A passing and not passing model for the property

In the LTS shown in Figure B.3a, in every possible trace A occurs. However, in the LTS shown in Figure B.3b, action A can occur, but it is also possible for the model to get stuck in a loop, resulting in A never occurring.

Property 4 (A at least n times). Parameters:

- Task A
- Number n

In every possible execution, task A occurs at least n times. If the model does not terminate, in each step either task A has occurred at least n times or it is possible for task A to occur in the future. If the model does terminate, task A occurs at least n before the model terminates.

Formula:

$$\nu X(c : \mathbb{N} := 0).((c \geq 5) \vee ([a]X(c+1) \wedge [\bar{a}]X(c) \wedge \mu Y.([\bar{a}]Y \wedge \langle true \rangle true)))$$

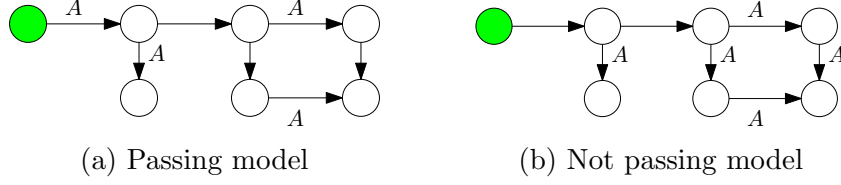


Figure B.4: A passing and not passing model for the property with $n = 2$

In the LTS shown in Figure B.4a, in every possible execution A occurs twice. However, in the LTS shown in Figure B.4b, action A can occur twice for some executions, but it is also possible for the model to reach a deadlock while A has only occurred once.

Property 5 (A at most n times). Parameters:

- Task A
- Number n

In every possible execution, task A occurs at most n times. When task A occurs, it cannot have occurred more than $n - 1$ times in the past.

Formula:

$$\nu X(c : \mathbb{N} := 0).([A]((c + 1 \leq n) \wedge X(c + 1)) \wedge [\bar{A}]X(c))$$

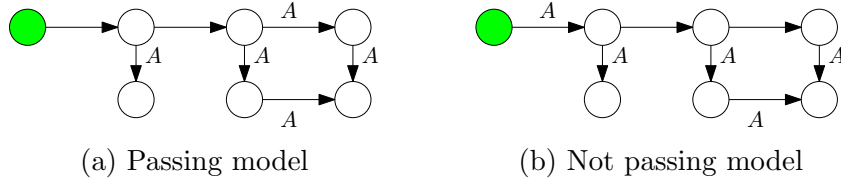


Figure B.5: A passing and not passing model for the property with $n = 2$

In the LTS shown in Figure B.5a, in every possible execution A occurs at most twice. However, in the LTS shown in Figure B.5b, action A can occur more than twice for some executions.

Property 6 (A exactly n times). Parameters:

- Task A
- Number n

In every possible execution, task A occurs exactly n times. If the model does not terminate, in each step either task A has occurred exactly n times or it is possible for task A to occur in the future, if task A has occurred

n times, task A can no longer occur. If the model does terminate, task A occurs exactly n before the model terminates.

Formula:

$$\begin{aligned} & \nu X(c : \mathbb{N} := 0). (\\ & \quad ((c < n) \wedge ([A]X(c + 1) \wedge [\bar{A}]X(c) \wedge \mu Y.([\bar{A}]Y \wedge \langle true \rangle true))) \vee \\ & \quad ((c \geq n) \wedge ([true^*][A]false)) \\ &) \end{aligned}$$

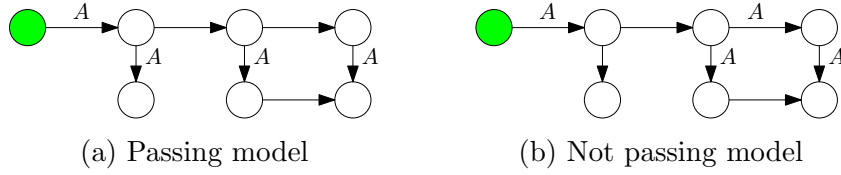


Figure B.6: A passing and not passing model for the property with $n = 2$

In the LTS shown in Figure B.6a, in every possible execution A occurs exactly twice. However, in the LTS shown in Figure B.6b, action A can occur more, or less than twice for some executions.

Property 7 (A at least as often as B). Parameters:

- Task A
- Task B

In every possible execution, task A occurs at least as often as task B . Whenever task B occurs, task A has occurred at least as often as task B .

Formula:

$$\begin{aligned} & \nu X(a : \mathbb{N} := 0, b : \mathbb{N} := 0). (\\ & \quad [A](X(a + 1, b)) \wedge \\ & \quad [B]((a \geq b) \wedge X(a, b + 1)) \wedge \\ & \quad [\overline{(A \cup B)}]X(a, b) \\ &) \end{aligned}$$

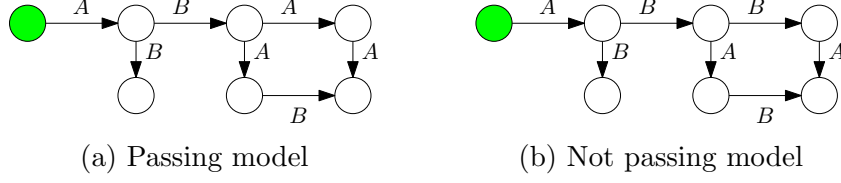


Figure B.7: A passing and not passing model for the property

In the LTS shown in Figure B.7a, in every possible execution whenever B occurs, A at least as often. However, in the LTS shown in Figure B.7b, the trace ABB can occur, violating the property.

Property 8 (A not before B). Parameters:

- Task A
- Task B

In every possible trace, task A cannot occur before task B has occurred. Whenever task A occurs, task B should have occurred.

Formula:

$$[\overline{B}^*][A]false$$

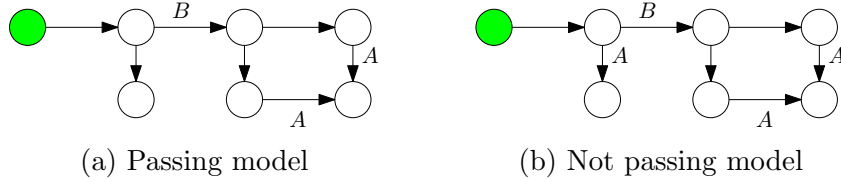


Figure B.8: A passing and not passing model for the property

In the LTS shown in Figure B.8a, in every possible execution whenever A occurs, B has occurred. However, in the LTS shown in Figure B.8b, it is possible for A to occur before B has occurred.

Property 9 (Q at most n items). Parameters:

- Queue Q
- Number n

In every possible execution, queue Q can have at most n items. It is not possible for queue Q to have more than n items.

Linear process equation:

$$\#S_Q > n \rightarrow raise_exception$$

Formula:

$$[true^*][raise_exception]false$$

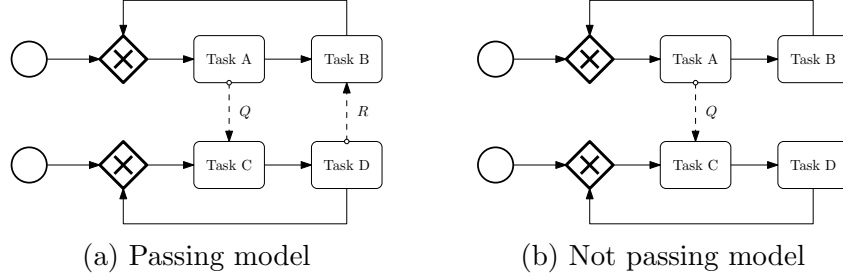


Figure B.9: A passing and not passing model for the property for $n = 1$

In the LTS shown in Figure B.9a, whenever task A occurs and puts an item in queue Q , it cannot occur again before task D puts an item in queue R , thus before a new item can be placed in queue Q , the previous item has to be removed. However, in the LTS shown in Figure B.9b, task A and B can occur endlessly, regardless of whether the items in queue Q are removed. Thus, an item can be placed in queue Q before the previous item is removed.

Property 10 (When A then eventually B). Parameters:

- Task A
- Task B

Whenever task A occurs, task B should eventually occur. If the model does not terminate, task B occurs in a finite number of steps, if the model does terminate, task B occurs before the model terminates.

Formula:

$$[true^*][A]\mu X.([\overline{B}]X \wedge \langle true \rangle true)$$

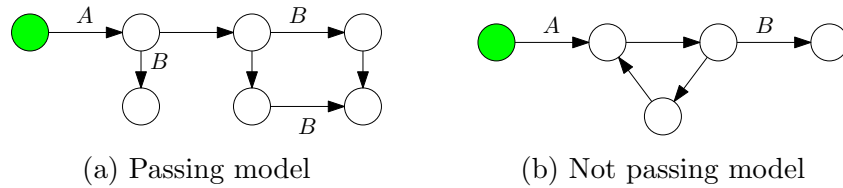


Figure B.10: A passing and not passing model for the property

In the LTS shown in Figure B.10a, whenever task A eventually task B occurs. However, in the LTS shown in Figure B.10b, it is possible that when task A occurs, the model gets stuck in an infinite loop, and task B never occurs.

Property 11 (A with $e=c$ ensures B before C). Parameters:

- Task A
- List of types e
- Constant c
- Task B
- Task C

Whenever task A occurs, and the expression e equals c , then if the model does not terminate, task B occurs in a finite number of steps and task B cannot occur in the meantime, if the model does terminate, task B occurs before the model terminates, and before task B possibly occurs.

Formula:

$$[true^*][A(c)]\mu X.([\overline{B}]X \wedge [C]false \wedge \langle true \rangle true)$$

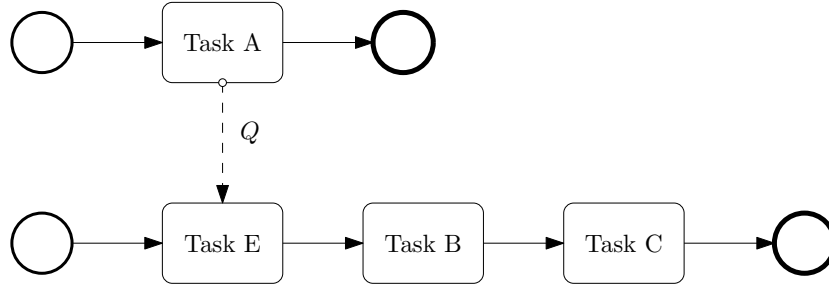


Figure B.11: Passing model

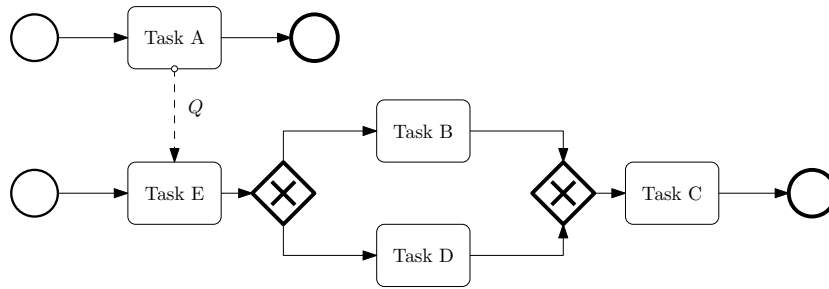


Figure B.12: Not passing model

In the LTS shown in Figure B.11, whenever task A occurs and puts an item in queue Q , task B has to occur before task C . However, in the LTS shown in Figure B.12, whenever task A occurs and puts an item in queue Q , it is possible for task D to occur followed by task C , violating the property.

Appendix C

Simulation

Simulation of a counterexample to $\mu X.([\overline{C}]X \wedge \langle true \rangle true)$, expressing that eventually C should occur, on the BPMN model a shown in Figure 7.8.

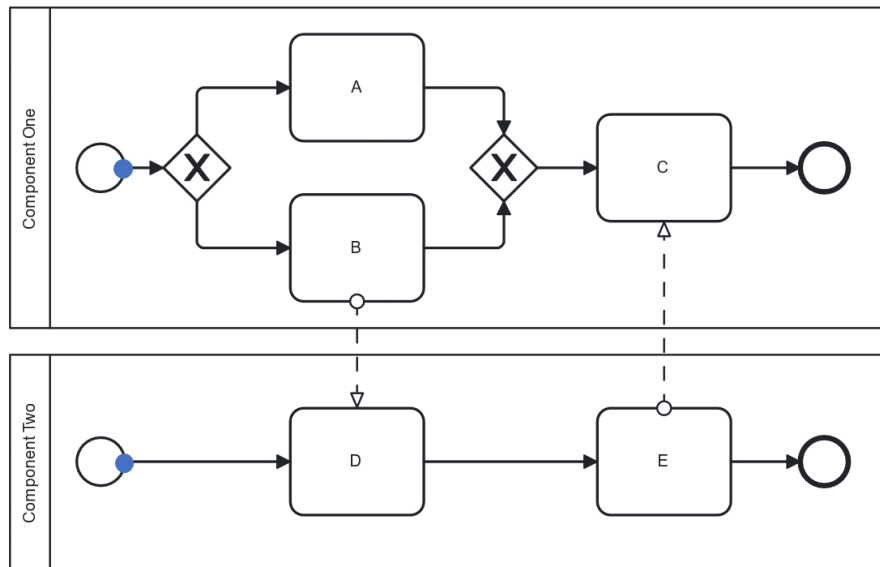


Figure C.1: Simulation initial state

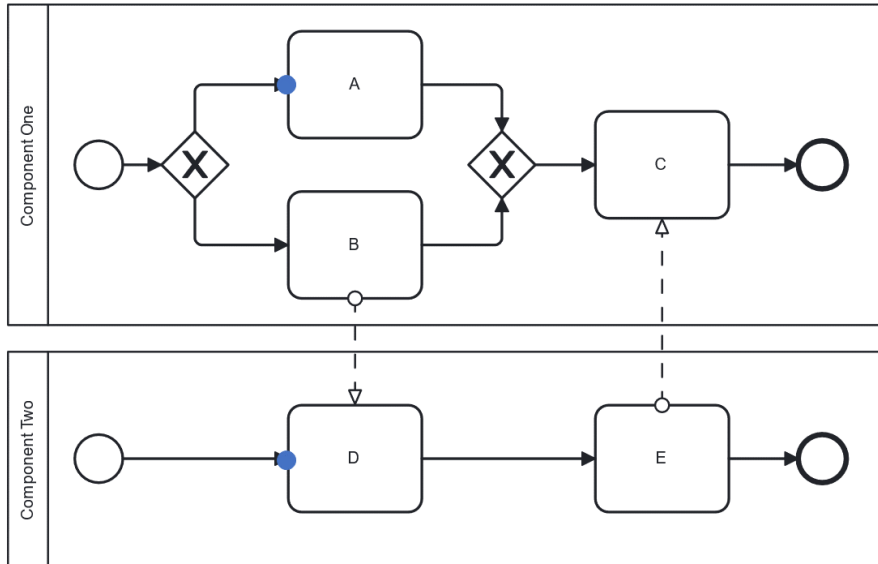


Figure C.2: Simulation before *A* occurring

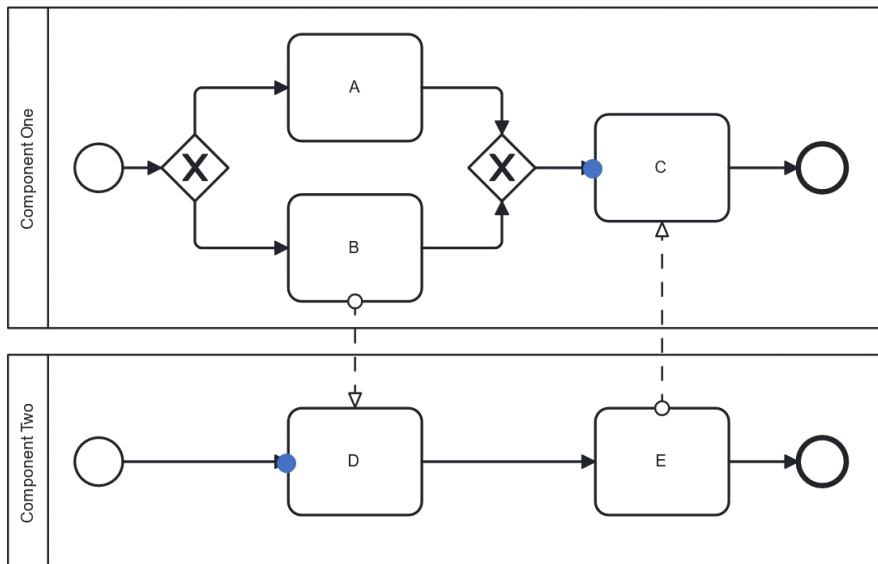


Figure C.3: Simulation after *A* occurring